

knn

April 3, 2026

```
[2]: # This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# TODO: Enter the foldername in your Drive where you have saved the unzipped
# assignment folder, e.g. 'cs231n/assignments/assignment1/'
FOLDERNAME = 'cs231n/assignments/assignment1/'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
%cd /content/drive/My\ Drive/$FOLDERNAME/cs231n/datasets/
!bash get_datasets.sh
%cd /content/drive/My\ Drive/$FOLDERNAME
```

```
Mounted at /content/drive
/content/drive/My Drive/cs231n/assignments/assignment1/cs231n/datasets
/content/drive/My Drive/cs231n/assignments/assignment1
```

1 k-Nearest Neighbor (kNN) exercise

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the [assignments page](#) on the course website.

The kNN classifier consists of two stages:

- During training, the classifier takes the training data and simply remembers it
- During testing, kNN classifies every test image by comparing to all training images and transferring the labels of the k most similar training examples
- The value of k is cross-validated

In this exercise you will implement these steps and understand the basic Image Classification pipeline, cross-validation, and gain proficiency in writing efficient, vectorized code.

```
[3]: import sys, IPython
      print(sys.version)
      print("IPython:", IPython.__version__)
```

3.11.13 (main, Jun 4 2025, 08:57:29) [GCC 11.4.0]

IPython: 7.34.0

```
[4]: # Run some setup code for this notebook.

import random
import numpy as np
from cs231n.data_utils import load_CIFAR10
import matplotlib.pyplot as plt

# This is a bit of magic to make matplotlib figures appear inline in the
↪notebook
# rather than in a new window.
%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# Some more magic so that the notebook will reload external python modules;
# see http://stackoverflow.com/questions/1907993/
↪autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2
```

```
[5]: # Load the raw CIFAR-10 data.
cifar10_dir = 'cs231n/datasets/cifar-10-batches-py'

# Cleaning up variables to prevent loading data multiple times (which may cause
↪memory issue)
try:
    del X_train, y_train
    del X_test, y_test
    print('Clear previously loaded data.')
except:
    pass

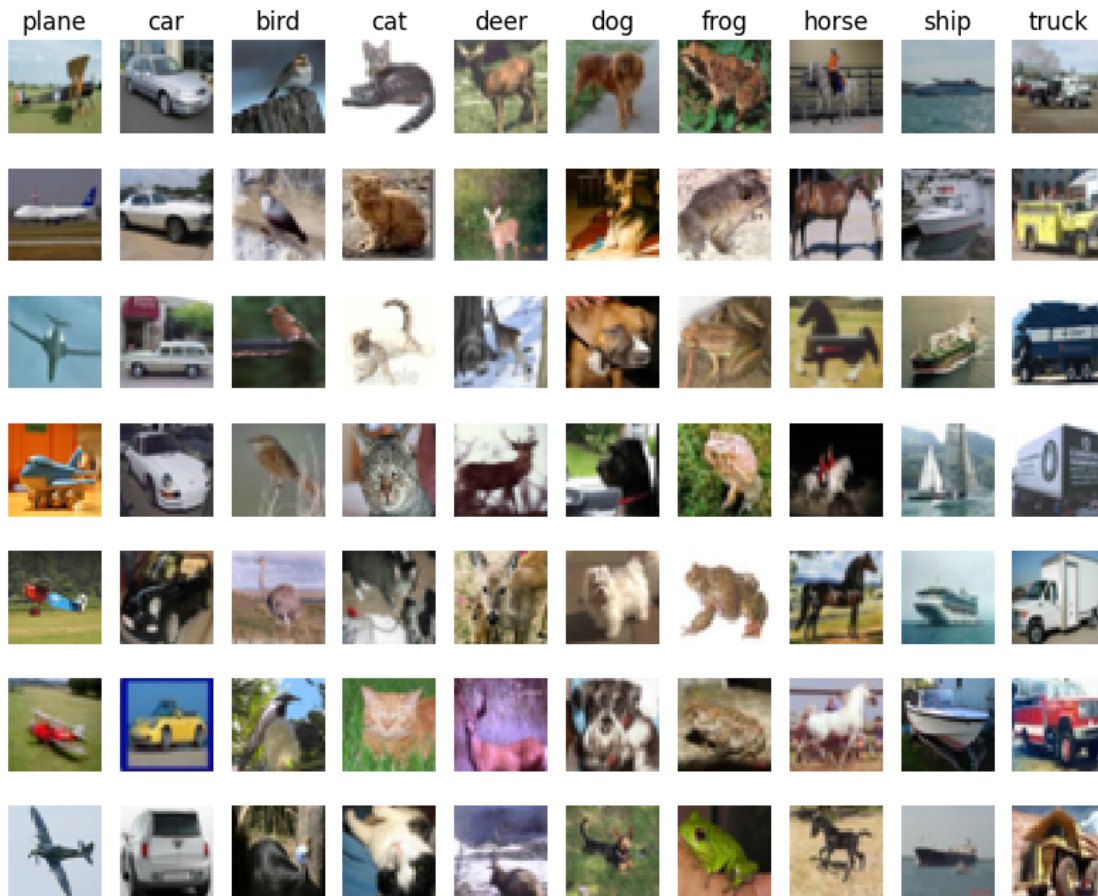
X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

# As a sanity check, we print out the size of the training and test data.
print('Training data shape: ', X_train.shape)
```

```
print('Training labels shape: ', y_train.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

```
Training data shape: (50000, 32, 32, 3)
Training labels shape: (50000,)
Test data shape: (10000, 32, 32, 3)
Test labels shape: (10000,)
```

```
[6]: # Visualize some examples from the dataset.
# We show a few examples of training images from each class.
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', '
↳ 'ship', 'truck']
num_classes = len(classes)
samples_per_class = 7
for y, cls in enumerate(classes):
    idxs = np.flatnonzero(y_train == y)
    idxs = np.random.choice(idxs, samples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt_idx = i * num_classes + y + 1
        plt.subplot(samples_per_class, num_classes, plt_idx)
        plt.imshow(X_train[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls)
plt.show()
```



```
[7]: # Subsample the data for more efficient code execution in this exercise
num_training = 5000
mask = list(range(num_training))
X_train = X_train[mask]
y_train = y_train[mask]

num_test = 500
mask = list(range(num_test))
X_test = X_test[mask]
y_test = y_test[mask]

# Reshape the image data into rows
X_train = np.reshape(X_train, (X_train.shape[0], -1))
X_test = np.reshape(X_test, (X_test.shape[0], -1))
print(X_train.shape, X_test.shape)
```

(5000, 3072) (500, 3072)

```
[8]: from cs231n.classifiers import KNearestNeighbor

# Create a kNN classifier instance.
# Remember that training a kNN classifier is a noop:
# the Classifier simply remembers the data and does no further processing
classifier = KNearestNeighbor()
classifier.train(X_train, y_train)
```

We would now like to classify the test data with the kNN classifier. Recall that we can break down this process into two steps:

1. First we must compute the distances between all test examples and all train examples.
2. Given these distances, for each test example we find the k nearest examples and have them vote for the label

Lets begin with computing the distance matrix between all training and test examples. For example, if there are N_{tr} training examples and N_{te} test examples, this stage should result in a $N_{te} \times N_{tr}$ matrix where each element (i,j) is the distance between the i-th test and j-th train example.

Note: For the three distance computations that we require you to implement in this notebook, you may not use the `np.linalg.norm()` function that numpy provides.

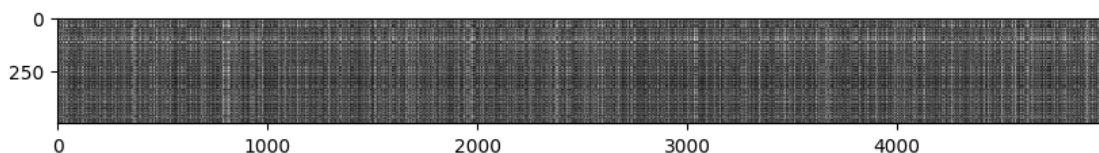
First, open `cs231n/classifiers/k_nearest_neighbor.py` and implement the function `compute_distances_two_loops` that uses a (very inefficient) double loop over all pairs of (test, train) examples and computes the distance matrix one element at a time.

```
[9]: # Open cs231n/classifiers/k_nearest_neighbor.py and implement
# compute_distances_two_loops.

# Test your implementation:
dists = classifier.compute_distances_two_loops(X_test)
print(dists.shape)
```

(500, 5000)

```
[10]: # We can visualize the distance matrix: each row is a single test example and
# its distances to training examples
plt.imshow(dists, interpolation='none')
plt.show()
```



Inline Question 1

Notice the structured patterns in the distance matrix, where some rows or columns are visibly brighter. (Note that with the default color scheme black indicates low distances while white indicates high distances.)

- What in the data is the cause behind the distinctly bright rows?
- What causes the columns?

Your Answer : outlier

```
[11]: # Now implement the function predict_labels and run the code below:
# We use k = 1 (which is Nearest Neighbor).
y_test_pred = classifier.predict_labels(dists, k=1)

# Compute and print the fraction of correctly predicted examples
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

Got 137 / 500 correct => accuracy: 0.274000

You should expect to see approximately 27% accuracy. Now let's try out a larger k, say k = 5:

```
[12]: y_test_pred = classifier.predict_labels(dists, k=5)
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

Got 139 / 500 correct => accuracy: 0.278000

You should expect to see a slightly better performance than with k = 1.

Inline Question 2

We can also use other distance metrics such as L1 distance. For pixel values $p_{ij}^{(k)}$ at location (i, j) of some image I_k ,

the mean μ across all pixels over all images is

$$\mu = \frac{1}{nhw} \sum_{k=1}^n \sum_{i=1}^h \sum_{j=1}^w p_{ij}^{(k)}$$

And the pixel-wise mean μ_{ij} across all images is

$$\mu_{ij} = \frac{1}{n} \sum_{k=1}^n p_{ij}^{(k)}.$$

The general standard deviation σ and pixel-wise standard deviation σ_{ij} is defined similarly.

Which of the following preprocessing steps will not change the performance of a Nearest Neighbor classifier that uses L1 distance? Select all that apply. To clarify, both training and test examples are preprocessed in the same way.

1. Subtracting the mean μ ($\tilde{p}_{ij}^{(k)} = p_{ij}^{(k)} - \mu$)

2. Subtracting the per pixel mean μ_{ij} ($\tilde{p}_{ij}^{(k)} = p_{ij}^{(k)} - \mu_{ij}$.)
3. Subtracting the mean μ and dividing by the standard deviation σ .
4. Subtracting the pixel-wise mean μ_{ij} and dividing by the pixel-wise standard deviation σ_{ij} .
5. Rotating the coordinate axes of the data, which means rotating all the images by the same angle. Empty regions in the image caused by rotation are padded with a same pixel value and no interpolation is performed.

Your Answer : 1,2,3,

Your Explanation : 4 causes some weight average; L1 is not rotation-invariant.

```
[15]: # Now lets speed up distance matrix computation by using partial vectorization
# with one loop. Implement the function compute_distances_one_loop and run the
# code below:
dists_one = classifier.compute_distances_one_loop(X_test)

# To ensure that our vectorized implementation is correct, we make sure that it
# agrees with the naive implementation. There are many ways to decide whether
# two matrices are similar; one of the simplest is the Frobenius norm. In case
# you haven't seen it before, the Frobenius norm of two matrices is the square
# root of the squared sum of differences of all elements; in other words,
# ↪reshape
# the matrices into vectors and compute the Euclidean distance between them.
difference = np.linalg.norm(dists - dists_one, ord='fro')
print('One loop difference was: %f' % (difference, ))
if difference < 0.001:
    print('Good! The distance matrices are the same')
else:
    print('Uh-oh! The distance matrices are different')
```

One loop difference was: 0.000000

Good! The distance matrices are the same

```
[16]: # Now implement the fully vectorized version inside compute_distances_no_loops
# and run the code
dists_two = classifier.compute_distances_no_loops(X_test)

# check that the distance matrix agrees with the one we computed before:
difference = np.linalg.norm(dists - dists_two, ord='fro')
print('No loop difference was: %f' % (difference, ))
if difference < 0.001:
    print('Good! The distance matrices are the same')
else:
    print('Uh-oh! The distance matrices are different')
```

No loop difference was: 0.000000

Good! The distance matrices are the same

```
[17]: # Let's compare how fast the implementations are
def time_function(f, *args):
    """
    Call a function f with args and return the time (in seconds) that it took
    to execute.
    """
    import time
    tic = time.time()
    f(*args)
    toc = time.time()
    return toc - tic

two_loop_time = time_function(classifier.compute_distances_two_loops, X_test)
print('Two loop version took %f seconds' % two_loop_time)

one_loop_time = time_function(classifier.compute_distances_one_loop, X_test)
print('One loop version took %f seconds' % one_loop_time)

no_loop_time = time_function(classifier.compute_distances_no_loops, X_test)
print('No loop version took %f seconds' % no_loop_time)

# You should see significantly faster performance with the fully vectorized
# implementation!

# NOTE: depending on what machine you're using,
# you might not see a speedup when you go from two loops to one loop,
# and might even see a slow-down.
```

```
Two loop version took 40.472754 seconds
One loop version took 53.025827 seconds
No loop version took 0.602363 seconds
```

1.0.1 Cross-validation

We have implemented the k-Nearest Neighbor classifier but we set the value $k = 5$ arbitrarily. We will now determine the best value of this hyperparameter with cross-validation.

```
[18]: num_folds = 5
k_choices = [1, 3, 5, 8, 10, 12, 15, 20, 50, 100]

X_train_folds = []
y_train_folds = []
#####
# TODO:
# Split up the training data into folds. After splitting, X_train_folds and
# y_train_folds should each be lists of length num_folds, where
# y_train_folds[i] is the label vector for the points in X_train_folds[i].
# Hint: Look up the numpy array_split function.
```



```
#####
X_train_folds = np.array_split(X_train, num_folds)
y_train_folds = np.array_split(y_train, num_folds)

# A dictionary holding the accuracies for different values of k that we find
# when running cross-validation. After running cross-validation,
# k_to_accuracies[k] should be a list of length num_folds giving the different
# accuracy values that we found when using that value of k.
k_to_accuracies = {}

#####
# TODO:
# Perform k-fold cross validation to find the best value of k. For each
# possible value of k, run the k-nearest-neighbor algorithm num_folds times,
# where in each case you use all but one of the folds as training data and the
# last fold as a validation set. Store the accuracies for all fold and all
# values of k in the k_to_accuracies dictionary.
#####
for k in k_choices:
    k_to_accuracies[k] = []
    for fold in range(num_folds):
        # 1) validation fold
        X_val = X_train_folds[fold]
        y_val = y_train_folds[fold]

        # 2) training folds (all except fold)
        X_tr = np.concatenate([X_train_folds[i] for i in range(num_folds) if i !=
        fold], axis=0)
        y_tr = np.concatenate([y_train_folds[i] for i in range(num_folds) if i !=
        fold], axis=0)

        # 3) train + predict
        classifier = KNearestNeighbor()
        classifier.train(X_tr, y_tr)
        y_val_pred = classifier.predict(X_val, k=k)

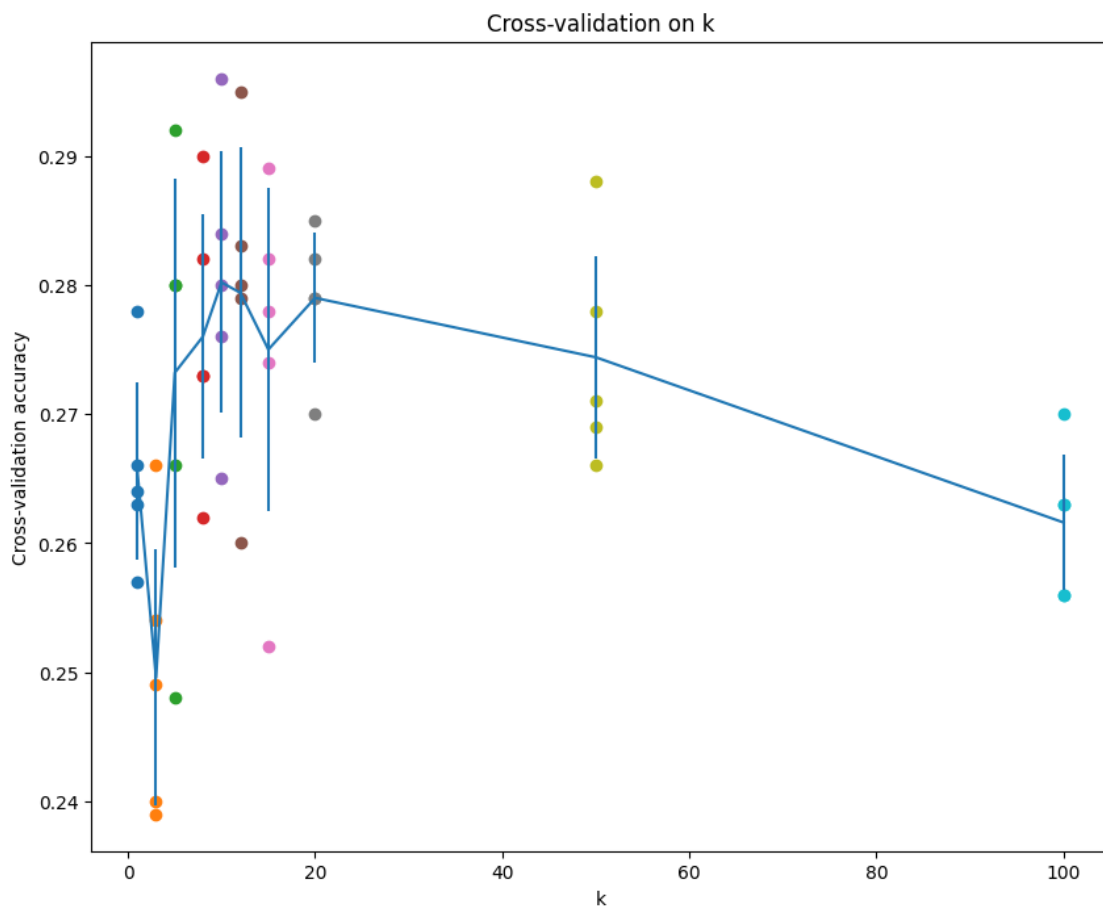
        # 4) accuracy
        acc = np.mean(y_val_pred == y_val)
        k_to_accuracies[k].append(acc)

# Print out the computed accuracies
for k in sorted(k_to_accuracies):
    for accuracy in k_to_accuracies[k]:
        print('k = %d, accuracy = %f' % (k, accuracy))
```

k = 1, accuracy = 0.263000
k = 1, accuracy = 0.257000
k = 1, accuracy = 0.264000
k = 1, accuracy = 0.278000
k = 1, accuracy = 0.266000
k = 3, accuracy = 0.239000
k = 3, accuracy = 0.249000
k = 3, accuracy = 0.240000
k = 3, accuracy = 0.266000
k = 3, accuracy = 0.254000
k = 5, accuracy = 0.248000
k = 5, accuracy = 0.266000
k = 5, accuracy = 0.280000
k = 5, accuracy = 0.292000
k = 5, accuracy = 0.280000
k = 8, accuracy = 0.262000
k = 8, accuracy = 0.282000
k = 8, accuracy = 0.273000
k = 8, accuracy = 0.290000
k = 8, accuracy = 0.273000
k = 10, accuracy = 0.265000
k = 10, accuracy = 0.296000
k = 10, accuracy = 0.276000
k = 10, accuracy = 0.284000
k = 10, accuracy = 0.280000
k = 12, accuracy = 0.260000
k = 12, accuracy = 0.295000
k = 12, accuracy = 0.279000
k = 12, accuracy = 0.283000
k = 12, accuracy = 0.280000
k = 15, accuracy = 0.252000
k = 15, accuracy = 0.289000
k = 15, accuracy = 0.278000
k = 15, accuracy = 0.282000
k = 15, accuracy = 0.274000
k = 20, accuracy = 0.270000
k = 20, accuracy = 0.279000
k = 20, accuracy = 0.279000
k = 20, accuracy = 0.282000
k = 20, accuracy = 0.285000
k = 50, accuracy = 0.271000
k = 50, accuracy = 0.288000
k = 50, accuracy = 0.278000
k = 50, accuracy = 0.269000
k = 50, accuracy = 0.266000
k = 100, accuracy = 0.256000
k = 100, accuracy = 0.270000
k = 100, accuracy = 0.263000

```
k = 100, accuracy = 0.256000  
k = 100, accuracy = 0.263000
```

```
[19]: # plot the raw observations  
for k in k_choices:  
    accuracies = k_to_accuracies[k]  
    plt.scatter([k] * len(accuracies), accuracies)  
  
# plot the trend line with error bars that correspond to standard deviation  
accuracies_mean = np.array([np.mean(v) for k,v in sorted(k_to_accuracies.  
    ↪items())])  
accuracies_std = np.array([np.std(v) for k,v in sorted(k_to_accuracies.  
    ↪items())])  
plt.errorbar(k_choices, accuracies_mean, yerr=accuracies_std)  
plt.title('Cross-validation on k')  
plt.xlabel('k')  
plt.ylabel('Cross-validation accuracy')  
plt.show()
```



```
[20]: # Based on the cross-validation results above, choose the best value for k,
# retrain the classifier using all the training data, and test it on the test
# data. You should be able to get above 28% accuracy on the test data.
best_k = 1

classifier = KNearestNeighbor()
classifier.train(X_train, y_train)
y_test_pred = classifier.predict(X_test, k=best_k)

# Compute and display the accuracy
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

Got 137 / 500 correct => accuracy: 0.274000

Inline Question 3

Which of the following statements about k -Nearest Neighbor (k -NN) are true in a classification setting, and for all k ? Select all that apply. 1. The decision boundary of the k -NN classifier is linear. 2. The training error of a 1-NN will always be lower than or equal to that of 5-NN. 3. The test error of a 1-NN will always be lower than that of a 5-NN. 4. The time needed to classify a test example with the k -NN classifier grows with the size of the training set. 5. None of the above.

Your Answer : 2,4

Your Explanation : 3:overfitting for 1-NN

softmax

April 3, 2026

```
[2]: # This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# TODO: Enter the foldername in your Drive where you have saved the unzipped
# assignment folder, e.g. 'cs231n/assignments/assignment1/'
FOLDERNAME = 'cs231n/assignments/assignment1/'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
%cd /content/drive/My\ Drive/$FOLDERNAME/cs231n/datasets/
!bash get_datasets.sh
%cd /content/drive/My\ Drive/$FOLDERNAME
```

Mounted at /content/drive
/content/drive/My Drive/cs231n/assignments/assignment1/cs231n/datasets
/content/drive/My Drive/cs231n/assignments/assignment1

1 Softmax Classifier exercise

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the [assignments page](#) on the course website.

In this exercise you will:

- implement a fully-vectorized **loss function** for the Softmax classifier.
- implement the fully-vectorized expression for its **analytic gradient**
- **check your implementation** using numerical gradient
- use a validation set to **tune the learning rate and regularization** strength
- **optimize** the loss function with **SGD**
- **visualize** the final learned weights

```
[3]: # Run some setup code for this notebook.
import random
import numpy as np
from cs231n.data_utils import load_CIFAR10
import matplotlib.pyplot as plt

# This is a bit of magic to make matplotlib figures appear inline in the
# notebook rather than in a new window.
%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# Some more magic so that the notebook will reload external python modules;
# see http://stackoverflow.com/questions/1907993/
  ↳ autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2
```

1.1 CIFAR-10 Data Loading and Preprocessing

```
[4]: # Load the raw CIFAR-10 data.
cifar10_dir = 'cs231n/datasets/cifar-10-batches-py'

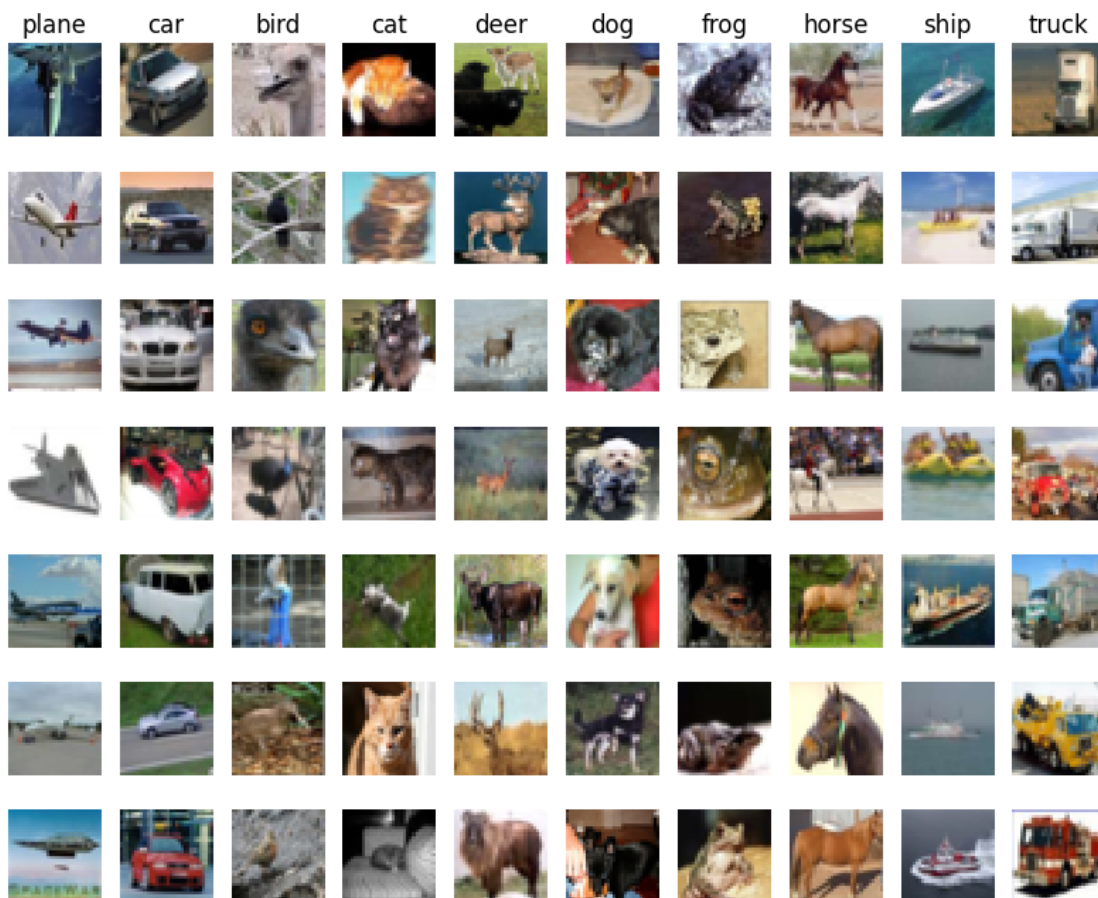
# Cleaning up variables to prevent loading data multiple times (which may cause
  ↳ memory issue)
try:
    del X_train, y_train
    del X_test, y_test
    print('Clear previously loaded data.')
except:
    pass

X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

# As a sanity check, we print out the size of the training and test data.
print('Training data shape: ', X_train.shape)
print('Training labels shape: ', y_train.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

```
Training data shape: (50000, 32, 32, 3)
Training labels shape: (50000,)
Test data shape: (10000, 32, 32, 3)
Test labels shape: (10000,)
```

```
[5]: # Visualize some examples from the dataset.
# We show a few examples of training images from each class.
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
num_classes = len(classes)
samples_per_class = 7
for y, cls in enumerate(classes):
    idxs = np.flatnonzero(y_train == y)
    idxs = np.random.choice(idxs, samples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt_idx = i * num_classes + y + 1
        plt.subplot(samples_per_class, num_classes, plt_idx)
        plt.imshow(X_train[idx].astype('uint8'))
        plt.axis('off')
    if i == 0:
        plt.title(cls)
plt.show()
```



```
[6]: # Split the data into train, val, and test sets. In addition we will
# create a small development set as a subset of the training data;
# we can use this for development so our code runs faster.
num_training = 49000
num_validation = 1000
num_test = 1000
num_dev = 500

# Our validation set will be num_validation points from the original
# training set.
mask = range(num_training, num_training + num_validation)
X_val = X_train[mask]
y_val = y_train[mask]

# Our training set will be the first num_train points from the original
# training set.
mask = range(num_training)
X_train = X_train[mask]
y_train = y_train[mask]

# We will also make a development set, which is a small subset of
# the training set.
mask = np.random.choice(num_training, num_dev, replace=False)
X_dev = X_train[mask]
y_dev = y_train[mask]

# We use the first num_test points of the original test set as our
# test set.
mask = range(num_test)
X_test = X_test[mask]
y_test = y_test[mask]

print('Train data shape: ', X_train.shape)
print('Train labels shape: ', y_train.shape)
print('Validation data shape: ', X_val.shape)
print('Validation labels shape: ', y_val.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

```
Train data shape: (49000, 32, 32, 3)
Train labels shape: (49000,)
Validation data shape: (1000, 32, 32, 3)
Validation labels shape: (1000,)
Test data shape: (1000, 32, 32, 3)
Test labels shape: (1000,)
```



```
[7]: # Preprocessing: reshape the image data into rows
X_train = np.reshape(X_train, (X_train.shape[0], -1))
X_val = np.reshape(X_val, (X_val.shape[0], -1))
X_test = np.reshape(X_test, (X_test.shape[0], -1))
X_dev = np.reshape(X_dev, (X_dev.shape[0], -1))

# As a sanity check, print out the shapes of the data
print('Training data shape: ', X_train.shape)
print('Validation data shape: ', X_val.shape)
print('Test data shape: ', X_test.shape)
print('dev data shape: ', X_dev.shape)
```

```
Training data shape: (49000, 3072)
Validation data shape: (1000, 3072)
Test data shape: (1000, 3072)
dev data shape: (500, 3072)
```

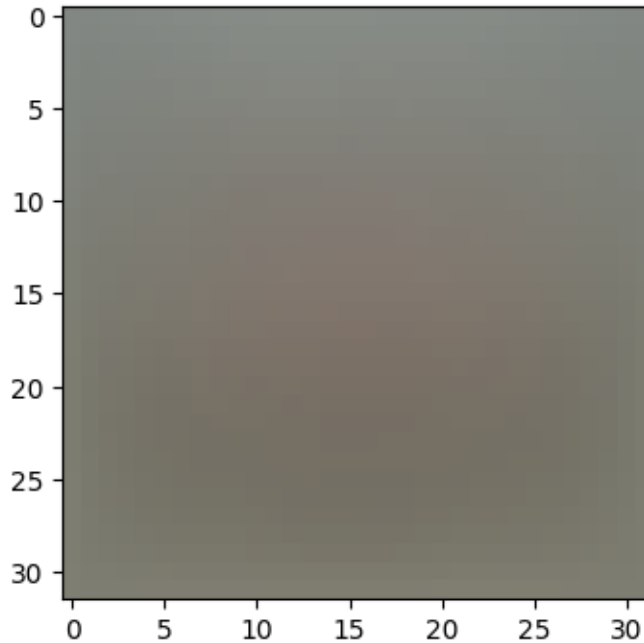
```
[8]: # Preprocessing: subtract the mean image
# first: compute the image mean based on the training data
mean_image = np.mean(X_train, axis=0)
print(mean_image[:10]) # print a few of the elements
plt.figure(figsize=(4,4))
plt.imshow(mean_image.reshape((32,32,3)).astype('uint8')) # visualize the mean_
    ↪ image
plt.show()

# second: subtract the mean image from train and test data
X_train -= mean_image
X_val -= mean_image
X_test -= mean_image
X_dev -= mean_image

# third: append the bias dimension of ones (i.e. bias trick) so that our_
    ↪ classifier
# only has to worry about optimizing a single weight matrix W.
X_train = np.hstack([X_train, np.ones((X_train.shape[0], 1))])
X_val = np.hstack([X_val, np.ones((X_val.shape[0], 1))])
X_test = np.hstack([X_test, np.ones((X_test.shape[0], 1))])
X_dev = np.hstack([X_dev, np.ones((X_dev.shape[0], 1))])

print(X_train.shape, X_val.shape, X_test.shape, X_dev.shape)
```

```
[130.64189796 135.98173469 132.47391837 130.05569388 135.34804082
131.75402041 130.96055102 136.14328571 132.47636735 131.48467347]
```



(49000, 3073) (1000, 3073) (1000, 3073) (500, 3073)

1.2 Softmax Classifier

Your code for this section will all be written inside `cs231n/classifiers/softmax.py`.

As you can see, we have prefilled the function `softmax_loss_naive` which uses for loops to evaluate the softmax loss function.

```
[9]: # Evaluate the naive implementation of the loss we provided for you:
from cs231n.classifiers.softmax import softmax_loss_naive
import time

# generate a random Softmax classifier weight matrix of small numbers
W = np.random.randn(3073, 10) * 0.0001

loss, grad = softmax_loss_naive(W, X_dev, y_dev, 0.000005)
print('loss: %f' % (loss, ))

# As a rough sanity check, our loss should be something close to -log(0.1).
print('loss: %f' % loss)
print('sanity check: %f' % (-np.log(0.1)))
```

loss: 2.334084

loss: 2.334084

sanity check: 2.302585

Inline Question 1

Why do we expect our loss to be close to $-\log(0.1)$? Explain briefly.**

Your Answer : Because with 10 classes, a randomly initialized classifier will often assign about equal probability to each class, so the correct class gets probability around 0.1.

The `grad` returned from the function above is right now all zero. Derive and implement the gradient for the softmax loss function and implement it inline inside the function `softmax_loss_naive`. You will find it helpful to interleave your new code inside the existing function.

To check that you have correctly implemented the gradient, you can numerically estimate the gradient of the loss function and compare the numeric estimate to the gradient that you computed. We have provided code that does this for you:

```
[10]: # Once you've implemented the gradient, recompute it with the code below
# and gradient check it with the function we provided for you

# Compute the loss and its gradient at W.
loss, grad = softmax_loss_naive(W, X_dev, y_dev, 0.0)

# Numerically compute the gradient along several randomly chosen dimensions, and
# compare them with your analytically computed gradient. The numbers should
# match
# almost exactly along all dimensions.
from cs231n.gradient_check import grad_check_sparse
f = lambda w: softmax_loss_naive(w, X_dev, y_dev, 0.0)[0]
grad_numerical = grad_check_sparse(f, W, grad)

# do the gradient check once again with regularization turned on
# you didn't forget the regularization gradient did you?
loss, grad = softmax_loss_naive(W, X_dev, y_dev, 5e1)
f = lambda w: softmax_loss_naive(w, X_dev, y_dev, 5e1)[0]
grad_numerical = grad_check_sparse(f, W, grad)
```

```
numerical: 1.267954 analytic: 1.267954, relative error: 1.093945e-08
numerical: 0.520412 analytic: 0.520412, relative error: 1.543531e-08
numerical: -2.837460 analytic: -2.837460, relative error: 2.221347e-09
numerical: 1.471233 analytic: 1.471233, relative error: 4.552930e-08
numerical: 1.120715 analytic: 1.120715, relative error: 6.453014e-08
numerical: -0.777718 analytic: -0.777718, relative error: 2.131637e-08
numerical: -2.287317 analytic: -2.287317, relative error: 3.497402e-09
numerical: 1.350149 analytic: 1.350149, relative error: 3.540670e-09
numerical: 1.553327 analytic: 1.553327, relative error: 2.714491e-08
numerical: 0.686740 analytic: 0.686740, relative error: 2.475149e-08
numerical: -1.966479 analytic: -1.966479, relative error: 9.433362e-09
numerical: -0.024705 analytic: -0.024705, relative error: 1.143124e-06
numerical: -1.564082 analytic: -1.564082, relative error: 5.903559e-09
numerical: -0.592683 analytic: -0.592683, relative error: 1.115055e-08
numerical: 1.613329 analytic: 1.613329, relative error: 1.788680e-08
```

```
numerical: 0.560462 analytic: 0.560462, relative error: 1.076701e-07
numerical: 1.750833 analytic: 1.750833, relative error: 2.811460e-08
numerical: 0.410068 analytic: 0.410068, relative error: 3.191322e-08
numerical: -0.684463 analytic: -0.684463, relative error: 2.574250e-08
numerical: -1.281497 analytic: -1.281497, relative error: 3.692452e-09
```

Inline Question 2

Although gradcheck is reliable softmax loss, it is possible that for SVM loss, once in a while, a dimension in the gradcheck will not match exactly. What could such a discrepancy be caused by? Is it a reason for concern? What is a simple example in one dimension where a svm loss gradient check could fail? How would change the margin affect of the frequency of this happening?

Note that SVM loss for a sample (x_i, y_i) is defined as:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + \Delta)$$

where j iterates over all classes except the correct class y_i and s_j denotes the classifier score for j^{th} class. Δ is a scalar margin. For more information, refer to ‘Multiclass Support Vector Machine loss’ on [this](#) page.

Hint: the SVM loss function is not strictly speaking differentiable.

Your Answer : SVM loss is not differentiable at the hinge point.

```
[13]: # Next implement the function softmax_loss_vectorized; for now only compute the
      ↪ loss;
      # we will implement the gradient in a moment.
      tic = time.time()
      loss_naive, grad_naive = softmax_loss_naive(W, X_dev, y_dev, 0.000005)
      toc = time.time()
      print('Naive loss: %e computed in %fs' % (loss_naive, toc - tic))

      from cs231n.classifiers.softmax import softmax_loss_vectorized
      tic = time.time()
      loss_vectorized, _ = softmax_loss_vectorized(W, X_dev, y_dev, 0.000005)
      toc = time.time()
      print('Vectorized loss: %e computed in %fs' % (loss_vectorized, toc - tic))

      # The losses should match but your vectorized implementation should be much
      ↪ faster.
      print('difference: %f' % (loss_naive - loss_vectorized))
```

```
Naive loss: 2.334084e+00 computed in 0.068470s
Vectorized loss: 2.334084e+00 computed in 0.012105s
difference: -0.000000
```

```
[14]: # Complete the implementation of softmax_loss_vectorized, and compute the
      ↪ gradient
      # of the loss function in a vectorized way.
```

```

# The naive implementation and the vectorized implementation should match, but
# the vectorized version should still be much faster.
tic = time.time()
_, grad_naive = softmax_loss_naive(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Naive loss and gradient: computed in %fs' % (toc - tic))

tic = time.time()
_, grad_vectorized = softmax_loss_vectorized(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Vectorized loss and gradient: computed in %fs' % (toc - tic))

# The loss is a single number, so it is easy to compare the values computed
# by the two implementations. The gradient on the other hand is a matrix, so
# we use the Frobenius norm to compare them.
difference = np.linalg.norm(grad_naive - grad_vectorized, ord='fro')
print('difference: %f' % difference)

```

```

Naive loss and gradient: computed in 0.113020s
Vectorized loss and gradient: computed in 0.018987s
difference: 0.000000

```

1.2.1 Stochastic Gradient Descent

We now have vectorized and efficient expressions for the loss, the gradient and our gradient matches the numerical gradient. We are therefore ready to do SGD to minimize the loss. Your code for this part will be written inside `cs231n/classifiers/linear_classifier.py`.

```

[15]: # In the file linear_classifier.py, implement SGD in the function
# LinearClassifier.train() and then run it with the code below.
from cs231n.classifiers import Softmax
softmax = Softmax()
tic = time.time()
loss_hist = softmax.train(X_train, y_train, learning_rate=1e-7, reg=2.5e4,
                          num_iters=1500, verbose=True)
toc = time.time()
print('That took %fs' % (toc - tic))

```

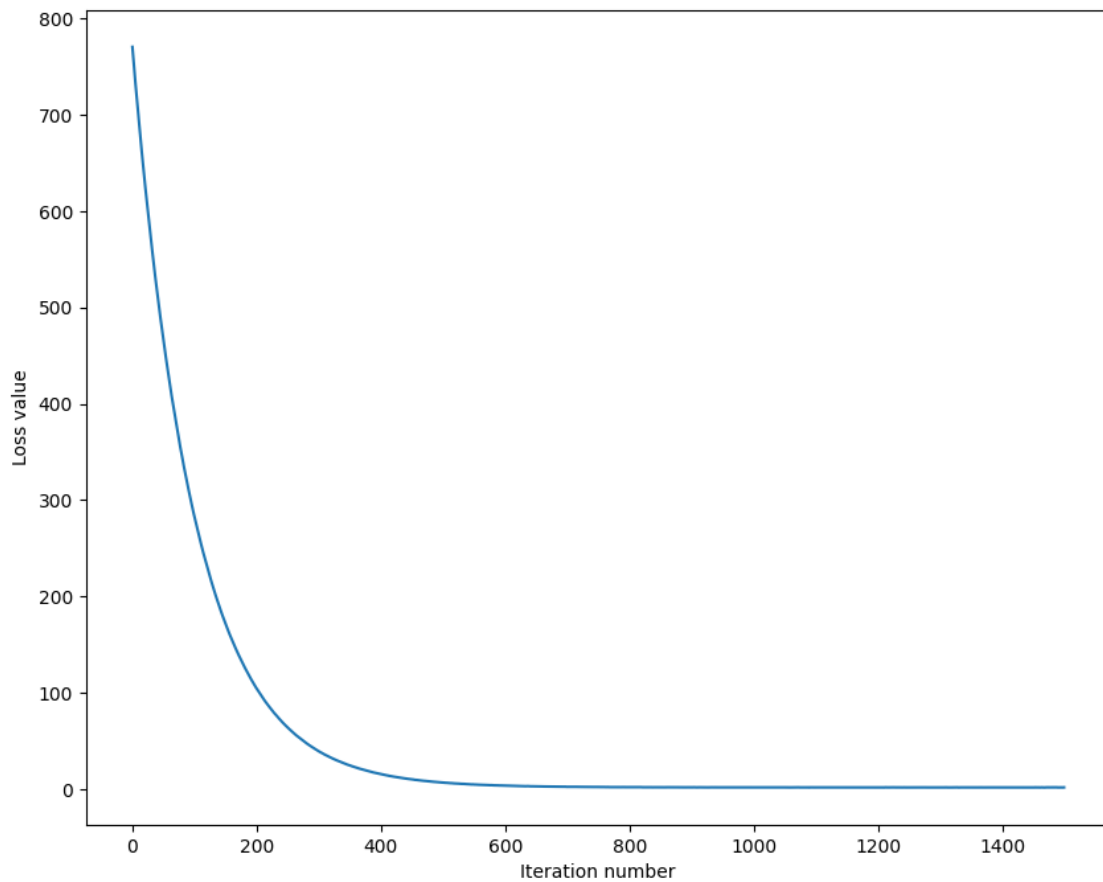
```

iteration 0 / 1500: loss 770.138794
iteration 100 / 1500: loss 282.424268
iteration 200 / 1500: loss 104.663013
iteration 300 / 1500: loss 39.559997
iteration 400 / 1500: loss 15.792491
iteration 500 / 1500: loss 7.108229
iteration 600 / 1500: loss 3.972675
iteration 700 / 1500: loss 2.692346
iteration 800 / 1500: loss 2.325991

```

```
iteration 900 / 1500: loss 2.122762
iteration 1000 / 1500: loss 2.129223
iteration 1100 / 1500: loss 2.115067
iteration 1200 / 1500: loss 2.107919
iteration 1300 / 1500: loss 2.133561
iteration 1400 / 1500: loss 2.064516
That took 10.757788s
```

```
[16]: # A useful debugging strategy is to plot the loss as a function of
# iteration number:
plt.plot(loss_hist)
plt.xlabel('Iteration number')
plt.ylabel('Loss value')
plt.show()
```



```
[17]: # Write the LinearClassifier.predict function and evaluate the performance on
# both the training and validation set
# You should get validation accuracy of about 0.34 (> 0.33).
y_train_pred = softmax.predict(X_train)
```

```
print('training accuracy: %f' % (np.mean(y_train == y_train_pred), ))
y_val_pred = softmax.predict(X_val)
print('validation accuracy: %f' % (np.mean(y_val == y_val_pred), ))
```

training accuracy: 0.100265
validation accuracy: 0.087000

```
[18]: # Save the trained model for autograder.
softmax.save("softmax.npy")
```

softmax.npy saved.

```
[19]: # Use the validation set to tune hyperparameters (regularization strength and
# learning rate). You should experiment with different ranges for the learning
# rates and regularization strengths; if you are careful you should be able to
# get a classification accuracy of about 0.365 (> 0.36) on the validation set.

# Note: you may see runtime/overflow warnings during hyper-parameter search.
# This may be caused by extreme values, and is not a bug.

# results is dictionary mapping tuples of the form
# (learning_rate, regularization_strength) to tuples of the form
# (training_accuracy, validation_accuracy). The accuracy is simply the fraction
# of data points that are correctly classified.
results = {}
best_val = -1 # The highest validation accuracy that we have seen so far.
best_softmax = None # The Softmax object that achieved the highest validation
    ↪rate.

#####
# TODO:
# Write code that chooses the best hyperparameters by tuning on the validation #
# set. For each combination of hyperparameters, train a Softmax on the. #
# training set, compute its accuracy on the training and validation sets, and #
# store these numbers in the results dictionary. In addition, store the best #
# validation accuracy in best_val and the Softmax object that achieves this. #
# accuracy in best_softmax. #
# #
# Hint: You should use a small value for num_iters as you develop your #
# validation code so that the classifiers don't take much time to train; once #
# you are confident that your validation code works, you should rerun the #
# code with a larger value for num_iters. #
#####

# Provided as a reference. You may or may not want to change these
    ↪hyperparameters
learning_rates = [1e-7, 1e-6]
```

```

regularization_strengths = [2.5e4, 1e4]

for lr in learning_rates:
    for reg in regularization_strengths:

        softmax = Softmax()
        softmax.train(
            X_train,
            y_train,
            learning_rate=lr,
            reg=reg,
            num_iters=1500,
            batch_size=200,
            verbose=False
        )

        y_train_pred = softmax.predict(X_train)
        y_val_pred = softmax.predict(X_val)

        train_accuracy = np.mean(y_train == y_train_pred)
        val_accuracy = np.mean(y_val == y_val_pred)

        results[(lr, reg)] = (train_accuracy, val_accuracy)

        if val_accuracy > best_val:
            best_val = val_accuracy
            best_softmax = softmax

# Print out results.
for lr, reg in sorted(results):
    train_accuracy, val_accuracy = results[(lr, reg)]
    print('lr %e reg %e train accuracy: %f val accuracy: %f' % (
        lr, reg, train_accuracy, val_accuracy))

print('best validation accuracy achieved during cross-validation: %f' %
      best_val)

```

```

lr 1.000000e-07 reg 1.000000e+04 train accuracy: 0.100265 val accuracy: 0.087000
lr 1.000000e-07 reg 2.500000e+04 train accuracy: 0.100265 val accuracy: 0.087000
lr 1.000000e-06 reg 1.000000e+04 train accuracy: 0.100265 val accuracy: 0.087000
lr 1.000000e-06 reg 2.500000e+04 train accuracy: 0.100265 val accuracy: 0.087000
best validation accuracy achieved during cross-validation: 0.087000

```

```

[20]: # Visualize the cross-validation results
import math
import pdb

```



```

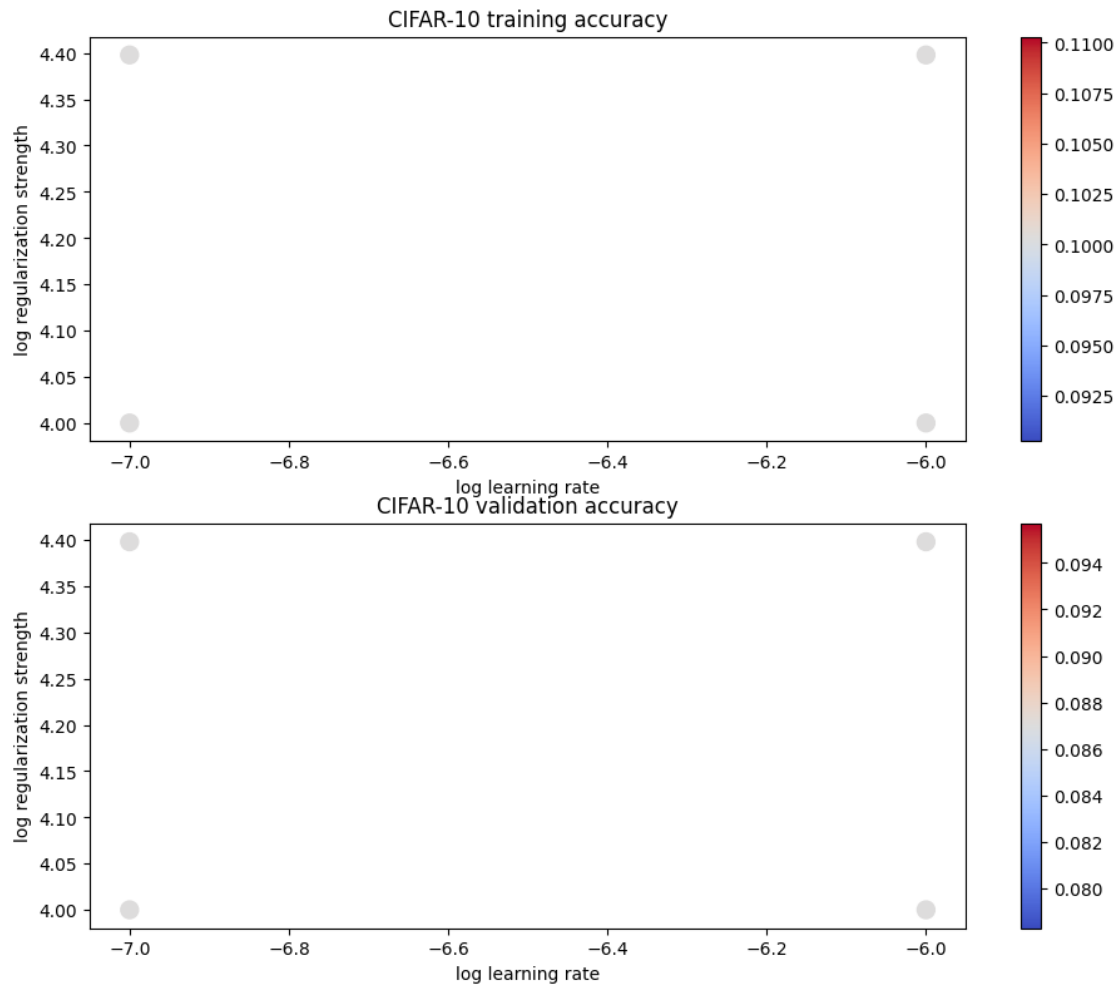
# pdb.set_trace()

x_scatter = [math.log10(x[0]) for x in results]
y_scatter = [math.log10(x[1]) for x in results]

# plot training accuracy
marker_size = 100
colors = [results[x][0] for x in results]
plt.subplot(2, 1, 1)
plt.tight_layout(pad=3)
plt.scatter(x_scatter, y_scatter, marker_size, c=colors, cmap=plt.cm.coolwarm)
plt.colorbar()
plt.xlabel('log learning rate')
plt.ylabel('log regularization strength')
plt.title('CIFAR-10 training accuracy')

# plot validation accuracy
colors = [results[x][1] for x in results] # default size of markers is 20
plt.subplot(2, 1, 2)
plt.scatter(x_scatter, y_scatter, marker_size, c=colors, cmap=plt.cm.coolwarm)
plt.colorbar()
plt.xlabel('log learning rate')
plt.ylabel('log regularization strength')
plt.title('CIFAR-10 validation accuracy')
plt.show()

```



```
[21]: # Evaluate the best softmax on test set
y_test_pred = best_softmax.predict(X_test)
test_accuracy = np.mean(y_test == y_test_pred)
print('Softmax classifier on raw pixels final test set accuracy: %f' % test_accuracy)
```

Softmax classifier on raw pixels final test set accuracy: 0.103000

```
[22]: # Save best softmax model
best_softmax.save("best_softmax.npy")
```

best_softmax.npy saved.

```
[23]: # Visualize the learned weights for each class.
# Depending on your choice of learning rate and regularization strength, these
# may
# or may not be nice to look at.
```

```

w = best_softmax.W[:-1,:] # strip out the bias
w = w.reshape(32, 32, 3, 10)
w_min, w_max = np.min(w), np.max(w)
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
for i in range(10):
    plt.subplot(2, 5, i + 1)

    # Rescale the weights to be between 0 and 255
    wimg = 255.0 * (w[:, :, :, i].squeeze() - w_min) / (w_max - w_min)
    plt.imshow(wimg.astype('uint8'))
    plt.axis('off')
    plt.title(classes[i])

```



Inline question 3

Describe what your visualized Softmax classifier weights look like, and offer a brief explanation for why they look the way they do.

Your Answer : The Softmax weights usually look like blurry prototypes of each class. For each class, the weight image often highlights the colors and coarse shapes that are most useful for identifying that category.

Inline Question 4 - True or False

Suppose the overall training loss is defined as the sum of the per-datapoint loss over all training

examples. It is possible to add a new datapoint to a training set that would change the softmax loss, but leave the SVM loss unchanged.

Your Answer : True

Your Explanation : If the correct class score is larger than every incorrect class score by at least γ , then every term is zero, so that datapoint adds no SVM loss

two_layer_net

April 3, 2026

```
[ ]: # This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# TODO: Enter the foldername in your Drive where you have saved the unzipped
# assignment folder, e.g. 'cs231n/assignments/assignment1/'
FOLDERNAME = 'cs231n/assignments/assignment1/'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
%cd /content/drive/My\ Drive/$FOLDERNAME/cs231n/datasets/
!bash get_datasets.sh
%cd /content/drive/My\ Drive/$FOLDERNAME
```

Mounted at /content/drive
/content/drive/My Drive/cs231n/assignments/assignment1/cs231n/datasets
/content/drive/My Drive/cs231n/assignments/assignment1

1 Fully-Connected Neural Nets

In this exercise we will implement fully-connected networks using a modular approach. For each layer we will implement a **forward** and a **backward** function. The **forward** function will receive inputs, weights, and other parameters and will return both an output and a **cache** object storing data needed for the backward pass, like this:

```
def layer_forward(x, w):
    """ Receive inputs x and weights w """
    # Do some computations ...
    z = # ... some intermediate value
    # Do some more computations ...
    out = # the output
```

```
cache = (x, w, z, out) # Values we need to compute gradients
```

```
return out, cache
```

The backward pass will receive upstream derivatives and the `cache` object, and will return gradients with respect to the inputs and weights, like this:

```
def layer_backward(dout, cache):
    """
    Receive dout (derivative of loss with respect to outputs) and cache,
    and compute derivative with respect to inputs.
    """
    # Unpack cache values
    x, w, z, out = cache

    # Use values in cache to compute derivatives
    dx = # Derivative of loss with respect to x
    dw = # Derivative of loss with respect to w

    return dx, dw
```

After implementing a bunch of layers this way, we will be able to easily combine them to build classifiers with different architectures.

```
[ ]: # As usual, a bit of setup
from __future__ import print_function
import time
import numpy as np
import matplotlib.pyplot as plt
from cs231n.classifiers.fc_net import *
from cs231n.data_utils import get_CIFAR10_data
from cs231n.gradient_check import eval_numerical_gradient, \
    eval_numerical_gradient_array
from cs231n.solver import Solver

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# for auto-reloading external modules
# see http://stackoverflow.com/questions/1907993/
# autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2

def rel_error(x, y):
    """ returns relative error """
```

```
return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))
```

```
[ ]: # Load the (preprocessed) CIFAR10 data.
```

```
data = get_CIFAR10_data()
for k, v in list(data.items()):
    print('%s: ' % k, v.shape)
```

```
('X_train: ', (49000, 3, 32, 32))
('y_train: ', (49000,))
('X_val: ', (1000, 3, 32, 32))
('y_val: ', (1000,))
('X_test: ', (1000, 3, 32, 32))
('y_test: ', (1000,))
```

2 Affine layer: forward

Open the file `cs231n/layers.py` and implement the `affine_forward` function.

Once you are done you can test your implementation by running the following:

```
[ ]: # Test the affine_forward function
```

```
num_inputs = 2
input_shape = (4, 5, 6)
output_dim = 3

input_size = num_inputs * np.prod(input_shape)
weight_size = output_dim * np.prod(input_shape)

x = np.linspace(-0.1, 0.5, num=input_size).reshape(num_inputs, *input_shape)
w = np.linspace(-0.2, 0.3, num=weight_size).reshape(np.prod(input_shape),
    ↪output_dim)
b = np.linspace(-0.3, 0.1, num=output_dim)

out, _ = affine_forward(x, w, b)
correct_out = np.array([[ 1.49834967,  1.70660132,  1.91485297],
                        [ 3.25553199,  3.5141327,   3.77273342]])

# Compare your output with ours. The error should be around e-9 or less.
print('Testing affine_forward function:')
print('difference: ', rel_error(out, correct_out))
```

```
Testing affine_forward function:
difference: 9.769848888397517e-10
```

3 Affine layer: backward

Now implement the `affine_backward` function and test your implementation using numeric gradient checking.

```
[ ]: # Test the affine_backward function
np.random.seed(231)
x = np.random.randn(10, 2, 3)
w = np.random.randn(6, 5)

b = np.random.randn(5)
dout = np.random.randn(10, 5)

dx_num = eval_numerical_gradient_array(lambda x: affine_forward(x, w, b)[0], x,
    ↪dout)
dw_num = eval_numerical_gradient_array(lambda w: affine_forward(x, w, b)[0], w,
    ↪dout)
db_num = eval_numerical_gradient_array(lambda b: affine_forward(x, w, b)[0], b,
    ↪dout)

_, cache = affine_forward(x, w, b)
dx, dw, db = affine_backward(dout, cache)

# The error should be around e-10 or less
print('Testing affine_backward function:')
print('dx error: ', rel_error(dx_num, dx))
print('dw error: ', rel_error(dw_num, dw))
print('db error: ', rel_error(db_num, db))
```

```
Testing affine_backward function:
dx error:  5.399100368651805e-11
dw error:  9.904211865398145e-11
db error:  2.4122867568119087e-11
```

4 ReLU activation: forward

Implement the forward pass for the ReLU activation function in the `relu_forward` function and test your implementation using the following:

```
[ ]: # Test the relu_forward function

x = np.linspace(-0.5, 0.5, num=12).reshape(3, 4)

out, _ = relu_forward(x)
correct_out = np.array([[ 0.,          0.,          0.,          0.],
                        [ 0.,          0.,          0.04545455, 0.13636364],
                        [ 0.22727273, 0.31818182, 0.40909091, 0.5]])
```



```
# Compare your output with ours. The error should be on the order of e-8
print('Testing relu_forward function:')
print('difference: ', rel_error(out, correct_out))
```

```
Testing relu_forward function:
difference: 4.999999798022158e-08
```

5 ReLU activation: backward

Now implement the backward pass for the ReLU activation function in the `relu_backward` function and test your implementation using numeric gradient checking:

```
[ ]: np.random.seed(231)
x = np.random.randn(10, 10)
dout = np.random.randn(*x.shape)

dx_num = eval_numerical_gradient_array(lambda x: relu_forward(x)[0], x, dout)

_, cache = relu_forward(x)
dx = relu_backward(dout, cache)

# The error should be on the order of e-12
print('Testing relu_backward function:')
print('dx error: ', rel_error(dx_num, dx))
```

```
Testing relu_backward function:
dx error: 3.2756349136310288e-12
```

5.1 Inline Question 1:

We’ve only asked you to implement ReLU, but there are a number of different activation functions that one could use in neural networks, each with its pros and cons. In particular, an issue commonly seen with activation functions is getting zero (or close to zero) gradient flow during backpropagation. Which of the following activation functions have this problem? If you consider these functions in the one dimensional case, what types of input would lead to this behaviour? 1. Sigmoid 2. ReLU 3. Leaky ReLU

Your Answer : 1 when x is large

6 “Sandwich” layers

There are some common patterns of layers that are frequently used in neural nets. For example, affine layers are frequently followed by a ReLU nonlinearity. To make these common patterns easy, we define several convenience layers in the file `cs231n/layer_utils.py`.

For now take a look at the `affine_relu_forward` and `affine_relu_backward` functions, and run the following to numerically gradient check the backward pass:

```
[ ]: from cs231n.layer_utils import affine_relu_forward, affine_relu_backward
np.random.seed(231)
x = np.random.randn(2, 3, 4)
w = np.random.randn(12, 10)
b = np.random.randn(10)
dout = np.random.randn(2, 10)

out, cache = affine_relu_forward(x, w, b)
dx, dw, db = affine_relu_backward(dout, cache)

dx_num = eval_numerical_gradient_array(lambda x: affine_relu_forward(x, w,
    ↪b)[0], x, dout)
dw_num = eval_numerical_gradient_array(lambda w: affine_relu_forward(x, w,
    ↪b)[0], w, dout)
db_num = eval_numerical_gradient_array(lambda b: affine_relu_forward(x, w,
    ↪b)[0], b, dout)

# Relative error should be around e-10 or less
print('Testing affine_relu_forward and affine_relu_backward:')
print('dx error: ', rel_error(dx_num, dx))
print('dw error: ', rel_error(dw_num, dw))
print('db error: ', rel_error(db_num, db))
```

```
Testing affine_relu_forward and affine_relu_backward:
dx error:  2.299579177309368e-11
dw error:  8.162011105764925e-11
db error:  7.826724021458994e-12
```

7 Loss layers: Softmax

Now implement the loss and gradient for softmax in the `softmax_loss` function in `cs231n/layers.py`. These should be similar to what you implemented in `cs231n/classifiers/softmax.py`. Other loss functions (e.g. `svm_loss`) can also be implemented in a modular way, however, it is not required for this assignment.

You can make sure that the implementations are correct by running the following:

```
[ ]: np.random.seed(231)
num_classes, num_inputs = 10, 50
x = 0.001 * np.random.randn(num_inputs, num_classes)
y = np.random.randint(num_classes, size=num_inputs)

dx_num = eval_numerical_gradient(lambda x: softmax_loss(x, y)[0], x,
    ↪verbose=False)
loss, dx = softmax_loss(x, y)
```

```
# Test softmax_loss function. Loss should be close to 2.3 and dx error should
    ↪ be around e-8
print('\nTesting softmax_loss:')
print('loss: ', loss)
print('dx error: ', rel_error(dx_num, dx))
```

```
Testing softmax_loss:
loss:  2.3025458445007376
dx error:  8.234144091578429e-09
```

8 Two-layer network

Open the file `cs231n/classifiers/fc_net.py` and complete the implementation of the `TwoLayerNet` class. Read through it to make sure you understand the API. You can run the cell below to test your implementation.

```
[14]: np.random.seed(231)
N, D, H, C = 3, 5, 50, 7
X = np.random.randn(N, D)
y = np.random.randint(C, size=N)

std = 1e-3
model = TwoLayerNet(input_dim=D, hidden_dim=H, num_classes=C, weight_scale=std)

print('Testing initialization ... ')
W1_std = abs(model.params['W1'].std() - std)
b1 = model.params['b1']
W2_std = abs(model.params['W2'].std() - std)
b2 = model.params['b2']
assert W1_std < std / 10, 'First layer weights do not seem right'
assert np.all(b1 == 0), 'First layer biases do not seem right'
assert W2_std < std / 10, 'Second layer weights do not seem right'
assert np.all(b2 == 0), 'Second layer biases do not seem right'

print('Testing test-time forward pass ... ')
model.params['W1'] = np.linspace(-0.7, 0.3, num=D*H).reshape(D, H)
model.params['b1'] = np.linspace(-0.1, 0.9, num=H)
model.params['W2'] = np.linspace(-0.3, 0.4, num=H*C).reshape(H, C)
model.params['b2'] = np.linspace(-0.9, 0.1, num=C)
X = np.linspace(-5.5, 4.5, num=N*D).reshape(D, N).T
scores = model.loss(X)
correct_scores = np.asarray(
    [[11.53165108, 12.2917344, 13.05181771, 13.81190102, 14.57198434, 15.
    ↪ 33206765, 16.09215096],
    [12.05769098, 12.74614105, 13.43459113, 14.1230412, 14.81149128, 15.
    ↪ 49994135, 16.18839143],
```

```

[12.58373087, 13.20054771, 13.81736455, 14.43418138, 15.05099822, 15.
↪66781506, 16.2846319 ]])
scores_diff = np.abs(scores - correct_scores).sum()
assert scores_diff < 1e-6, 'Problem with test-time forward pass'

print('Testing training loss (no regularization)')
y = np.asarray([0, 5, 1])
loss, grads = model.loss(X, y)
correct_loss = 3.4702243556
assert abs(loss - correct_loss) < 1e-10, 'Problem with training-time loss'

model.reg = 1.0
loss, grads = model.loss(X, y)
correct_loss = 26.5948426952
assert abs(loss - correct_loss) < 1e-10, 'Problem with regularization loss'

# Errors should be around e-7 or less
for reg in [0.0, 0.7]:
    print('Running numeric gradient check with reg = ', reg)
    model.reg = reg
    loss, grads = model.loss(X, y)

    for name in sorted(grads):
        f = lambda _: model.loss(X, y)[0]
        grad_num = eval_numerical_gradient(f, model.params[name], verbose=False)
        print('%s relative error: %.2e' % (name, rel_error(grad_num, grads[name])))

```

```

Testing initialization ...
Testing test-time forward pass ...
Testing training loss (no regularization)
Running numeric gradient check with reg = 0.0
W1 relative error: 1.52e-08
W2 relative error: 3.21e-10
b1 relative error: 8.37e-09
b2 relative error: 4.33e-10
Running numeric gradient check with reg = 0.7
W1 relative error: 2.53e-07
W2 relative error: 2.85e-08
b1 relative error: 1.56e-08
b2 relative error: 7.76e-10

```

```

[15]: from google.colab import drive
drive.mount('/content/drive')

```

Drive already mounted at /content/drive; to attempt to forcibly remount, call `drive.mount("/content/drive", force_remount=True)`.

9 Solver

Open the file `cs231n/solver.py` and read through it to familiarize yourself with the API. After doing so, use a `Solver` instance to train a `TwoLayerNet` that achieves about 36% accuracy on the validation set.

```
[16]: input_size = 32 * 32 * 3
hidden_size = 50
num_classes = 10
model = TwoLayerNet(input_size, hidden_size, num_classes)
solver = None

#####
# TODO: Use a Solver instance to train a TwoLayerNet that achieves about 36% #
# accuracy on the validation set.                                           #
#####
solver = Solver(
    model,
    data,
    update_rule='sgd',
    optim_config={
        'learning_rate': 1e-3,
    },
    lr_decay=0.95,
    num_epochs=10,
    batch_size=100,
    print_every=100,
)

solver.train()

#####
#                               END OF YOUR CODE                           #
#####
```

```
(Iteration 1 / 4900) loss: 2.300089
(Epoch 0 / 10) train acc: 0.171000; val_acc: 0.170000
(Iteration 101 / 4900) loss: 1.782419
(Iteration 201 / 4900) loss: 1.803466
(Iteration 301 / 4900) loss: 1.712676
(Iteration 401 / 4900) loss: 1.693946
(Epoch 1 / 10) train acc: 0.399000; val_acc: 0.428000
(Iteration 501 / 4900) loss: 1.711237
(Iteration 601 / 4900) loss: 1.443683
(Iteration 701 / 4900) loss: 1.574904
(Iteration 801 / 4900) loss: 1.526751
(Iteration 901 / 4900) loss: 1.352554
(Epoch 2 / 10) train acc: 0.463000; val_acc: 0.443000
```

(Iteration 1001 / 4900) loss: 1.340071
(Iteration 1101 / 4900) loss: 1.386843
(Iteration 1201 / 4900) loss: 1.489919
(Iteration 1301 / 4900) loss: 1.353077
(Iteration 1401 / 4900) loss: 1.467951
(Epoch 3 / 10) train acc: 0.477000; val_acc: 0.430000
(Iteration 1501 / 4900) loss: 1.337133
(Iteration 1601 / 4900) loss: 1.426819
(Iteration 1701 / 4900) loss: 1.348675
(Iteration 1801 / 4900) loss: 1.412626
(Iteration 1901 / 4900) loss: 1.354764
(Epoch 4 / 10) train acc: 0.497000; val_acc: 0.467000
(Iteration 2001 / 4900) loss: 1.422221
(Iteration 2101 / 4900) loss: 1.360665
(Iteration 2201 / 4900) loss: 1.546539
(Iteration 2301 / 4900) loss: 1.196704
(Iteration 2401 / 4900) loss: 1.401958
(Epoch 5 / 10) train acc: 0.508000; val_acc: 0.483000
(Iteration 2501 / 4900) loss: 1.243567
(Iteration 2601 / 4900) loss: 1.513808
(Iteration 2701 / 4900) loss: 1.266416
(Iteration 2801 / 4900) loss: 1.485956
(Iteration 2901 / 4900) loss: 1.049706
(Epoch 6 / 10) train acc: 0.533000; val_acc: 0.492000
(Iteration 3001 / 4900) loss: 1.264002
(Iteration 3101 / 4900) loss: 1.184786
(Iteration 3201 / 4900) loss: 1.231162
(Iteration 3301 / 4900) loss: 1.353725
(Iteration 3401 / 4900) loss: 1.217830
(Epoch 7 / 10) train acc: 0.545000; val_acc: 0.489000
(Iteration 3501 / 4900) loss: 1.446003
(Iteration 3601 / 4900) loss: 1.152495
(Iteration 3701 / 4900) loss: 1.421970
(Iteration 3801 / 4900) loss: 1.222752
(Iteration 3901 / 4900) loss: 1.213468
(Epoch 8 / 10) train acc: 0.546000; val_acc: 0.474000
(Iteration 4001 / 4900) loss: 1.187435
(Iteration 4101 / 4900) loss: 1.284799
(Iteration 4201 / 4900) loss: 1.135251
(Iteration 4301 / 4900) loss: 1.212217
(Iteration 4401 / 4900) loss: 1.213544
(Epoch 9 / 10) train acc: 0.586000; val_acc: 0.486000
(Iteration 4501 / 4900) loss: 1.306174
(Iteration 4601 / 4900) loss: 1.213528
(Iteration 4701 / 4900) loss: 1.220260
(Iteration 4801 / 4900) loss: 1.233231
(Epoch 10 / 10) train acc: 0.545000; val_acc: 0.483000

10 Debug the training

With the default parameters we provided above, you should get a validation accuracy of about 0.36 on the validation set. This isn't very good.

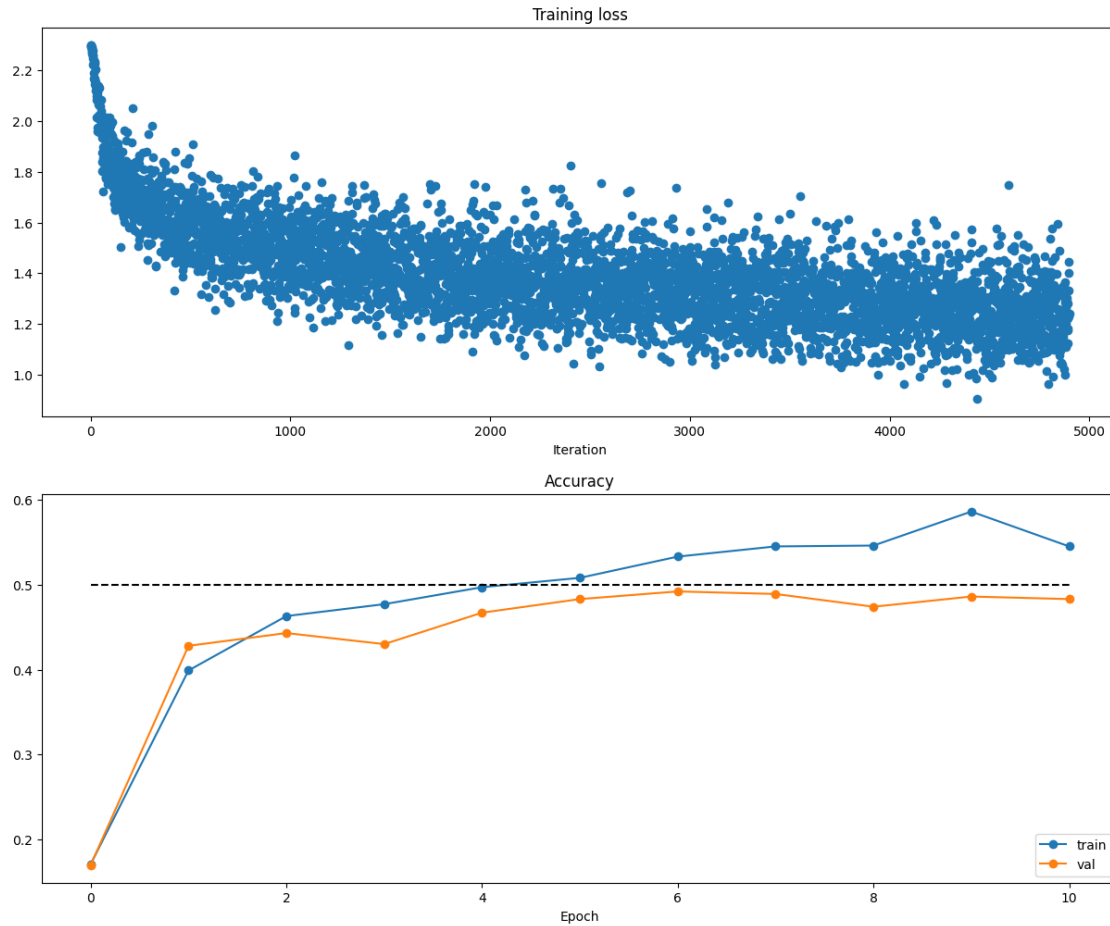
One strategy for getting insight into what's wrong is to plot the loss function and the accuracies on the training and validation sets during optimization.

Another strategy is to visualize the weights that were learned in the first layer of the network. In most neural networks trained on visual data, the first layer weights typically show some visible structure when visualized.

[17]: *# Run this cell to visualize training loss and train / val accuracy*

```
plt.subplot(2, 1, 1)
plt.title('Training loss')
plt.plot(solver.loss_history, 'o')
plt.xlabel('Iteration')

plt.subplot(2, 1, 2)
plt.title('Accuracy')
plt.plot(solver.train_acc_history, '-o', label='train')
plt.plot(solver.val_acc_history, '-o', label='val')
plt.plot([0.5] * len(solver.val_acc_history), 'k--')
plt.xlabel('Epoch')
plt.legend(loc='lower right')
plt.gcf().set_size_inches(15, 12)
plt.show()
```

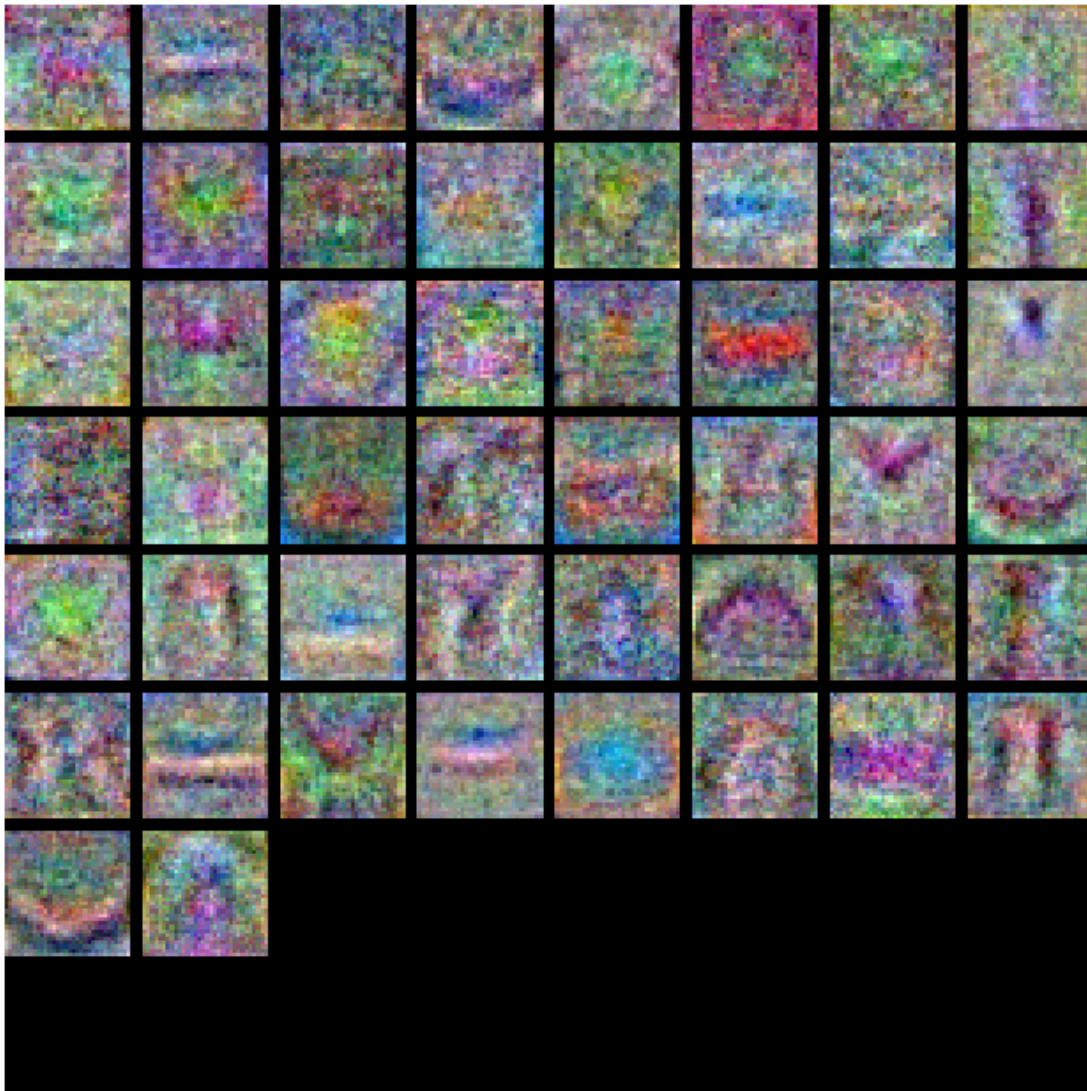


```
[18]: from cs231n.vis_utils import visualize_grid

# Visualize the weights of the network

def show_net_weights(net):
    W1 = net.params['W1']
    W1 = W1.reshape(3, 32, 32, -1).transpose(3, 1, 2, 0)
    plt.imshow(visualize_grid(W1, padding=3).astype('uint8'))
    plt.gca().axis('off')
    plt.show()

show_net_weights(model)
```

11 Tune your hyperparameters

What's wrong?. Looking at the visualizations above, we see that the loss is decreasing more or less linearly, which seems to suggest that the learning rate may be too low. Moreover, there is no gap between the training and validation accuracy, suggesting that the model we used has low capacity, and that we should increase its size. On the other hand, with a very large model we would expect to see more overfitting, which would manifest itself as a very large gap between the training and validation accuracy.

Tuning. Tuning the hyperparameters and developing intuition for how they affect the final performance is a large part of using Neural Networks, so we want you to get a lot of practice. Below, you should experiment with different values of the various hyperparameters, including hidden layer size, learning rate, number of training epochs, and regularization strength. You might also consider

tuning the learning rate decay, but you should be able to get good performance using the default value.

Approximate results. You should be aim to achieve a classification accuracy of greater than 48% on the validation set. Our best network gets over 52% on the validation set.

Experiment: Your goal in this exercise is to get as good of a result on CIFAR-10 as you can (52% could serve as a reference), with a fully-connected Neural Network. Feel free implement your own techniques (e.g. PCA to reduce dimensionality, or adding dropout, or adding features to the solver, etc.).

```
[ ]: best_model = None

#####
# TODO: Tune hyperparameters using the validation set. Store your best trained
# ↪#
# model in best_model.
# ↪#
#
# ↪#
# To help debug your network, it may help to use visualizations similar to the
# ↪#
# ones we used above; these visualizations will have significant qualitative
# ↪#
# differences from the ones we saw above for the poorly tuned network.
# ↪#
#
# ↪#
# Tweaking hyperparameters by hand can be fun, but you might find it useful to
# ↪#
# write code to sweep through possible combinations of hyperparameters
# ↪#
# automatically like we did on the previous exercises.
# ↪ #
#####
best_val = -1

learning_rates = [1e-3, 3e-3, 1e-2]
weight_scales = [1e-2, 5e-2, 1e-1]
regs = [0.0, 1e-3, 1e-2]

results = {}

for lr in learning_rates:
    for ws in weight_scales:
        for reg in regs:
```

```

    model = TwoLayerNet(
        input_dim=32 * 32 * 3,
        hidden_dim=100,
        num_classes=10,
        weight_scale=ws,
        reg=reg
    )

    solver = Solver(
        model,
        data,
        update_rule='sgd',
        optim_config={'learning_rate': lr},
        lr_decay=0.95,
        num_epochs=10,
        batch_size=100,
        print_every=100,
        verbose=False
    )

    solver.train()

    val_acc = solver.best_val_acc
    results[(lr, ws, reg)] = val_acc

    if val_acc > best_val:
        best_val = val_acc
        best_model = model

for k, v in sorted(results.items()):
    print(k, v)

print("best validation accuracy:", best_val)
#####
#                                     END OF YOUR CODE                                     #
#####

```

/content/drive/My Drive/cs231n/assignments/assignment1/cs231n/layers.py:733:

RuntimeWarning: divide by zero encountered in log

```
loss = -np.sum(np.log(probs[np.arange(N), y])) / N
```

/content/drive/My Drive/cs231n/assignments/assignment1/cs231n/layers.py:28:

RuntimeWarning: overflow encountered in matmul

```
out = x_row @ w + b
```

/content/drive/My Drive/cs231n/assignments/assignment1/cs231n/layers.py:728:

RuntimeWarning: overflow encountered in subtract

```
shifted = x - np.max(x, axis=1, keepdims=True)
```

```
/content/drive/My Drive/cs231n/assignments/assignment1/cs231n/layers.py:728:
```

```
RuntimeWarning: invalid value encountered in subtract
```

```
    shifted = x - np.max(x, axis=1, keepdims=True)
```

```
(0.001, 0.01, 0.0) 0.506
(0.001, 0.01, 0.001) 0.499
(0.001, 0.01, 0.01) 0.499
(0.001, 0.05, 0.0) 0.297
(0.001, 0.05, 0.001) 0.295
(0.001, 0.05, 0.01) 0.292
(0.001, 0.1, 0.0) 0.254
(0.001, 0.1, 0.001) 0.277
(0.001, 0.1, 0.01) 0.275
(0.003, 0.01, 0.0) 0.287
(0.003, 0.01, 0.001) 0.467
(0.003, 0.01, 0.01) 0.43
(0.003, 0.05, 0.0) 0.297
(0.003, 0.05, 0.001) 0.323
(0.003, 0.05, 0.01) 0.313
(0.003, 0.1, 0.0) 0.27
(0.003, 0.1, 0.001) 0.29
(0.003, 0.1, 0.01) 0.27
(0.01, 0.01, 0.0) 0.138
(0.01, 0.01, 0.001) 0.17
(0.01, 0.01, 0.01) 0.125
(0.01, 0.05, 0.0) 0.12
(0.01, 0.05, 0.001) 0.124
(0.01, 0.05, 0.01) 0.107
(0.01, 0.1, 0.0) 0.188
(0.01, 0.1, 0.001) 0.158
(0.01, 0.1, 0.01) 0.13
best validation accuracy: 0.506
```

12 Test your model!

Run your best model on the validation and test sets. You should achieve above 48% accuracy on the validation set and the test set.

```
[ ]: y_val_pred = np.argmax(best_model.loss(data['X_val']), axis=1)
      print('Validation set accuracy: ', (y_val_pred == data['y_val']).mean())
```

```
[ ]: y_test_pred = np.argmax(best_model.loss(data['X_test']), axis=1)
      print('Test set accuracy: ', (y_test_pred == data['y_test']).mean())
```

```
[ ]: # Save best model
      best_model.save("best_two_layer_net.npy")
```

12.1 Inline Question 2:

Now that you have trained a Neural Network classifier, you may find that your testing accuracy is much lower than the training accuracy. In what ways can we decrease this gap? Select all that apply.

1. Train on a larger dataset.
2. Add more hidden units.
3. Increase the regularization strength.
4. None of the above.

\$Your Answer:\$1 , 3

Your Explanation : 1: More training data usually helps the model learn patterns that generalize better

2: Adding more hidden units increases the model's capacity, which often makes overfitting worse

3: Stronger regularization helps constrain the model and reduces overfitting, which can shrink the train-test gap.

[]:

features

April 3, 2026

```
[3]: # This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# TODO: Enter the foldername in your Drive where you have saved the unzipped
# assignment folder, e.g. 'cs231n/assignments/assignment1/'
FOLDERNAME = 'cs231n/assignments/assignment1/'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
%cd /content/drive/My\ Drive/$FOLDERNAME/cs231n/datasets/
!bash get_datasets.sh
%cd /content/drive/My\ Drive/$FOLDERNAME
```

```
Mounted at /content/drive
/content/drive/My Drive/cs231n/assignments/assignment1/cs231n/datasets
/content/drive/My Drive/cs231n/assignments/assignment1
```

1 Image features exercise

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the [assignments page](#) on the course website.

We have seen that we can achieve reasonable performance on an image classification task by training a linear classifier on the pixels of the input image. In this exercise we will show that we can improve our classification performance by training linear classifiers not on raw pixels but on features that are computed from the raw pixels.

All of your work for this exercise will be done in this notebook.

```
[4]: import random
import numpy as np
from cs231n.data_utils import load_CIFAR10
import matplotlib.pyplot as plt

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# for auto-reloading external modules
# see http://stackoverflow.com/questions/1907993/
↳ autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2
```

1.1 Load data

Similar to previous exercises, we will load CIFAR-10 data from disk.

```
[5]: from cs231n.features import color_histogram_hsv, hog_feature

def get_CIFAR10_data(num_training=49000, num_validation=1000, num_test=1000):
    # Load the raw CIFAR-10 data
    cifar10_dir = 'cs231n/datasets/cifar-10-batches-py'

    # Cleaning up variables to prevent loading data multiple times (which may
    ↳ cause memory issue)
    try:
        del X_train, y_train
        del X_test, y_test
        print('Clear previously loaded data.')
    except:
        pass

    X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

    # Subsample the data
    mask = list(range(num_training, num_training + num_validation))
    X_val = X_train[mask]
    y_val = y_train[mask]
    mask = list(range(num_training))
    X_train = X_train[mask]
    y_train = y_train[mask]
    mask = list(range(num_test))
    X_test = X_test[mask]
```

```

    y_test = y_test[mask]

    return X_train, y_train, X_val, y_val, X_test, y_test

X_train, y_train, X_val, y_val, X_test, y_test = get_CIFAR10_data()

```

1.2 Extract Features

For each image we will compute a Histogram of Oriented Gradients (HOG) as well as a color histogram using the hue channel in HSV color space. We form our final feature vector for each image by concatenating the HOG and color histogram feature vectors.

Roughly speaking, HOG should capture the texture of the image while ignoring color information, and the color histogram represents the color of the input image while ignoring texture. As a result, we expect that using both together ought to work better than using either alone. Verifying this assumption would be a good thing to try for your own interest.

The `hog_feature` and `color_histogram_hsv` functions both operate on a single image and return a feature vector for that image. The `extract_features` function takes a set of images and a list of feature functions and evaluates each feature function on each image, storing the results in a matrix where each column is the concatenation of all feature vectors for a single image.

```

[6]: from cs231n.features import *

# num_color_bins = 10 # Number of bins in the color histogram
num_color_bins = 25 # Number of bins in the color histogram
feature_fns = [hog_feature, lambda img: color_histogram_hsv(img,
    ↪nbin=num_color_bins)]
X_train_feats = extract_features(X_train, feature_fns, verbose=True)
X_val_feats = extract_features(X_val, feature_fns)
X_test_feats = extract_features(X_test, feature_fns)

# Preprocessing: Subtract the mean feature
mean_feat = np.mean(X_train_feats, axis=0, keepdims=True)
X_train_feats -= mean_feat
X_val_feats -= mean_feat
X_test_feats -= mean_feat

# Preprocessing: Divide by standard deviation. This ensures that each feature
# has roughly the same scale.
std_feat = np.std(X_train_feats, axis=0, keepdims=True)
X_train_feats /= std_feat
X_val_feats /= std_feat
X_test_feats /= std_feat

# Preprocessing: Add a bias dimension
X_train_feats = np.hstack([X_train_feats, np.ones((X_train_feats.shape[0], 1))])
X_val_feats = np.hstack([X_val_feats, np.ones((X_val_feats.shape[0], 1))])

```



```
X_test_feats = np.hstack([X_test_feats, np.ones((X_test_feats.shape[0], 1))])
```

```
Done extracting features for 1000 / 49000 images
Done extracting features for 2000 / 49000 images
Done extracting features for 3000 / 49000 images
Done extracting features for 4000 / 49000 images
Done extracting features for 5000 / 49000 images
Done extracting features for 6000 / 49000 images
Done extracting features for 7000 / 49000 images
Done extracting features for 8000 / 49000 images
Done extracting features for 9000 / 49000 images
Done extracting features for 10000 / 49000 images
Done extracting features for 11000 / 49000 images
Done extracting features for 12000 / 49000 images
Done extracting features for 13000 / 49000 images
Done extracting features for 14000 / 49000 images
Done extracting features for 15000 / 49000 images
Done extracting features for 16000 / 49000 images
Done extracting features for 17000 / 49000 images
Done extracting features for 18000 / 49000 images
Done extracting features for 19000 / 49000 images
Done extracting features for 20000 / 49000 images
Done extracting features for 21000 / 49000 images
Done extracting features for 22000 / 49000 images
Done extracting features for 23000 / 49000 images
Done extracting features for 24000 / 49000 images
Done extracting features for 25000 / 49000 images
Done extracting features for 26000 / 49000 images
Done extracting features for 27000 / 49000 images
Done extracting features for 28000 / 49000 images
Done extracting features for 29000 / 49000 images
Done extracting features for 30000 / 49000 images
Done extracting features for 31000 / 49000 images
Done extracting features for 32000 / 49000 images
Done extracting features for 33000 / 49000 images
Done extracting features for 34000 / 49000 images
Done extracting features for 35000 / 49000 images
Done extracting features for 36000 / 49000 images
Done extracting features for 37000 / 49000 images
Done extracting features for 38000 / 49000 images
Done extracting features for 39000 / 49000 images
Done extracting features for 40000 / 49000 images
Done extracting features for 41000 / 49000 images
Done extracting features for 42000 / 49000 images
Done extracting features for 43000 / 49000 images
Done extracting features for 44000 / 49000 images
Done extracting features for 45000 / 49000 images
```

```
Done extracting features for 46000 / 49000 images
Done extracting features for 47000 / 49000 images
Done extracting features for 48000 / 49000 images
Done extracting features for 49000 / 49000 images
```

1.3 Train Softmax classifier on features

Using the Softmax code developed earlier in the assignment, train Softmax classifiers on top of the features extracted above; this should achieve better results than training them directly on top of raw pixels.

```
[7]: # Use the validation set to tune the learning rate and regularization strength

from cs231n.classifiers.linear_classifier import Softmax

learning_rates = [1e-7, 1e-6]
regularization_strengths = [5e5, 5e6]

results = {}
best_val = -1
best_softmax = None

#####
# TODO:
# Use the validation set to set the learning rate and regularization strength. #
# This should be identical to the validation that you did for the Softmax; save#
# the best trained classifier in best_softmax. If you carefully tune the model, #
# you should be able to get accuracy of above 0.42 on the validation set.      #
#####
for lr in learning_rates:
    for reg in regularization_strengths:
        softmax = Softmax()
        softmax.train(
            X_train_feats,
            y_train,
            learning_rate=lr,
            reg=reg,
            num_iters=2500,
            batch_size=300,
            verbose=False
        )

        y_train_pred = softmax.predict(X_train_feats)
        train_accuracy = np.mean(y_train == y_train_pred)

        y_val_pred = softmax.predict(X_val_feats)
        val_accuracy = np.mean(y_val == y_val_pred)
```

```

        results[(lr, reg)] = (train_accuracy, val_accuracy)

        if val_accuracy > best_val:
            best_val = val_accuracy
            best_softmax = softmax

# Print out results.
for lr, reg in sorted(results):
    train_accuracy, val_accuracy = results[(lr, reg)]
    print('lr %e reg %e train accuracy: %f val accuracy: %f' % (
        lr, reg, train_accuracy, val_accuracy))

print('best validation accuracy achieved: %f' % best_val)

```

```

/content/drive/My
Drive/cs231n/assignments/assignment1/cs231n/classifiers/softmax.py:93:
RuntimeWarning: divide by zero encountered in log
    loss = -np.sum(np.log(correct_class_probs)) / N
/content/drive/My
Drive/cs231n/assignments/assignment1/cs231n/classifiers/softmax.py:94:
RuntimeWarning: overflow encountered in scalar multiply
    loss += reg * np.sum(W * W)
/usr/local/lib/python3.11/dist-packages/numpy/_core/fromnumeric.py:86:
RuntimeWarning: overflow encountered in reduce
    return ufunc.reduce(obj, axis, dtype, out, **passkwargs)
/content/drive/My
Drive/cs231n/assignments/assignment1/cs231n/classifiers/softmax.py:94:
RuntimeWarning: overflow encountered in multiply
    loss += reg * np.sum(W * W)
/content/drive/My
Drive/cs231n/assignments/assignment1/cs231n/classifiers/softmax.py:102:
RuntimeWarning: overflow encountered in multiply
    dW += 2 * reg * W

lr 1.000000e-07 reg 5.000000e+05 train accuracy: 0.417265 val accuracy: 0.410000
lr 1.000000e-07 reg 5.000000e+06 train accuracy: 0.344306 val accuracy: 0.353000
lr 1.000000e-06 reg 5.000000e+05 train accuracy: 0.344061 val accuracy: 0.330000
lr 1.000000e-06 reg 5.000000e+06 train accuracy: 0.100265 val accuracy: 0.087000
best validation accuracy achieved: 0.410000

```

```

[8]: # Evaluate your trained Softmax on the test set: you should be able to get at
      ↪ least 0.42
y_test_pred = best_softmax.predict(X_test_feats)
test_accuracy = np.mean(y_test == y_test_pred)
print(test_accuracy)

```

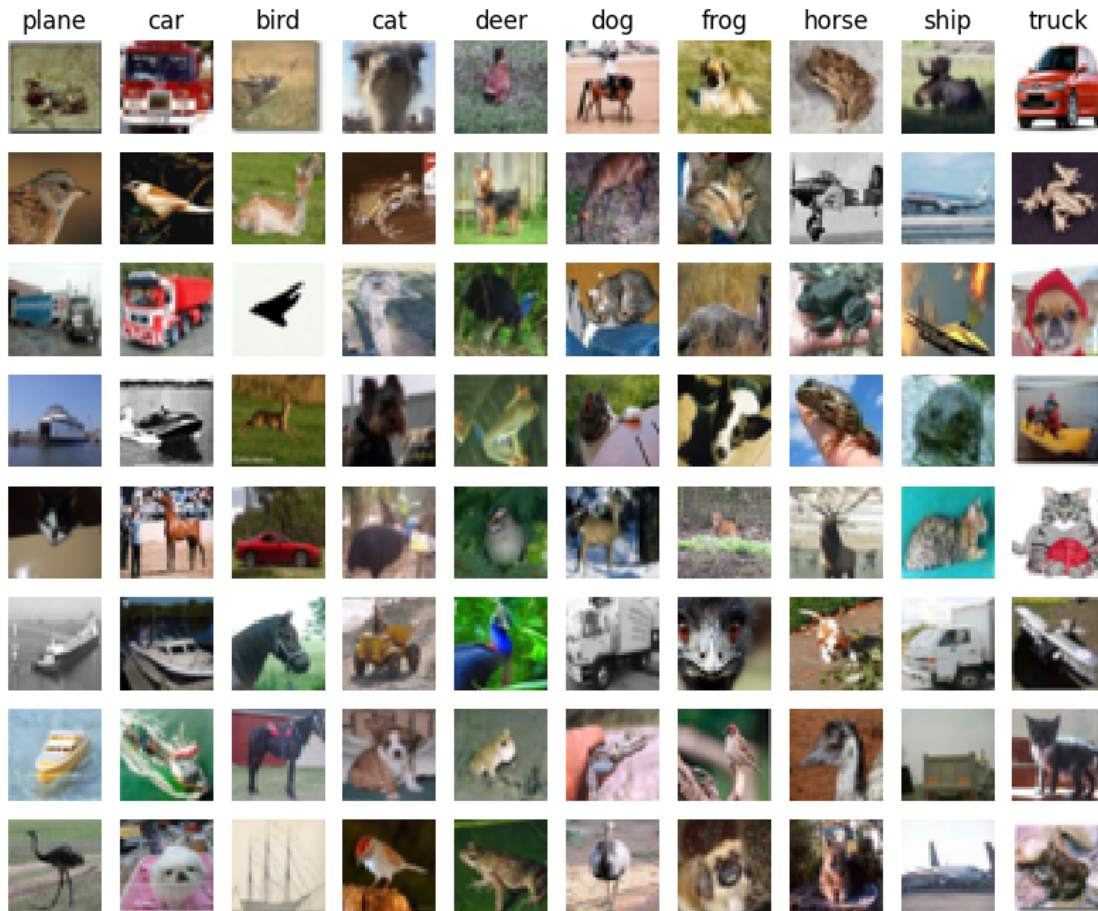
0.419

```
[9]: # Save best softmax model
best_softmax.save("best_softmax_features.npy")
```

best_softmax_features.npy saved.

```
[10]: # An important way to gain intuition about how an algorithm works is to
# visualize the mistakes that it makes. In this visualization, we show examples
# of images that are misclassified by our current system. The first column
# shows images that our system labeled as "plane" but whose true label is
# something other than "plane".

examples_per_class = 8
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',
           ↪ 'ship', 'truck']
for cls, cls_name in enumerate(classes):
    idxs = np.where((y_test != cls) & (y_test_pred == cls))[0]
    idxs = np.random.choice(idxs, examples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt.subplot(examples_per_class, len(classes), i * len(classes) + cls +
           ↪ 1)
        plt.imshow(X_test[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls_name)
plt.show()
```



1.3.1 Inline question 1:

Describe the misclassification results that you see. Do they make sense?

Your Answer : Most mistakes happen between categories with similar visual appearance, shape, color, or background

1.4 Neural Network on image features

Earlier in this assignment we saw that training a two-layer neural network on raw pixels achieved better classification performance than linear classifiers on raw pixels. In this notebook we have seen that linear classifiers on image features outperform linear classifiers on raw pixels.

For completeness, we should also try training a neural network on image features. This approach should outperform all previous approaches: you should easily be able to achieve over 55% classification accuracy on the test set; our best model achieves about 60% classification accuracy.

```
[11]: # Preprocessing: Remove the bias dimension
      # Make sure to run this cell only ONCE
      print(X_train_feats.shape)
```

```

X_train_feats = X_train_feats[:, :-1]
X_val_feats = X_val_feats[:, :-1]
X_test_feats = X_test_feats[:, :-1]

print(X_train_feats.shape)

```

```

(49000, 170)
(49000, 169)

```

```

[ ]: from cs231n.classifiers.fc_net import TwoLayerNet
      from cs231n.solver import Solver

input_dim = X_train_feats.shape[1]
hidden_dim = 500
num_classes = 10

data = {
    'X_train': X_train_feats,
    'y_train': y_train,
    'X_val': X_val_feats,
    'y_val': y_val,
    'X_test': X_test_feats,
    'y_test': y_test,
}

net = TwoLayerNet(input_dim, hidden_dim, num_classes)
best_net = None

learning_rates = [1e-3, 3e-3, 1e-2]
weight_scales = [1e-2]
regs = [1e-4, 1e-3, 1e-2]

for lr in learning_rates:
    for ws in weight_scales:
        for reg in regs:
            solver = Solver(
                net,
                data,
                update_rule='sgd',
                optim_config={'learning_rate': lr},
                lr_decay=0.95,
                num_epochs=25,
                batch_size=200,
                print_every=100,
                verbose=False
            )

```

```

solver.train()

train_acc = solver.train_acc_history[-1]
val_acc = solver.best_val_acc
results[(lr, ws, reg)] = (train_acc, val_acc)

if val_acc > best_val:
    best_val = val_acc
    best_net = net
#####
# TODO: Train a two-layer neural network on image features. You may want to #
# cross-validate various parameters as in previous sections. Store your best #
# model in the best_net variable. #
#####

```

```

[13]: # Run your best neural net classifier on the test set. You should be able
# to get more than 58% accuracy. It is also possible to get >60% accuracy
# with careful tuning.

```

```

y_test_pred = np.argmax(best_net.loss(data['X_test']), axis=1)
test_acc = (y_test_pred == data['y_test']).mean()
print(test_acc)

```

0.522

```

[ ]: # Save best model
best_net.save("best_two_layer_net_features.npy")

```

```

[ ]:

```

FullyConnectedNets

April 3, 2026

```
[ ]: # This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# TODO: Enter the foldername in your Drive where you have saved the unzipped
# assignment folder, e.g. 'cs231n/assignments/assignment1/'
FOLDERNAME = 'cs231n/assignments/assignment1/'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
%cd /content/drive/My\ Drive/$FOLDERNAME/cs231n/datasets/
# %cd /content/drive/My\ Drive/$FOLDERNAME
!bash get_datasets.sh
%cd /content/drive/My\ Drive/$FOLDERNAME
```

```
Mounted at /content/drive
/content/drive/My Drive/cs231n/assignments/assignment1/cs231n/datasets
/content/drive/My Drive/cs231n/assignments/assignment1
```

1 Multi-Layer Fully Connected Network

In this exercise, you will implement a fully connected network with an arbitrary number of hidden layers.

```
[ ]: # from google.colab import drive
# drive.mount('/content/drive')
```

Read through the `FullyConnectedNet` class in the file `cs231n/classifiers/fc_net.py`.

Implement the network initialization, forward pass, and backward pass. Throughout this assignment, you will be implementing layers in `cs231n/layers.py`. You can re-use your implementations

for `affine_forward`, `affine_backward`, `relu_forward`, `relu_backward`, and `softmax_loss` from before. For right now, don't worry about implementing dropout or batch/layer normalization yet, as you will add those features later.

```
[3]: # Setup cell.
import time
import numpy as np
import matplotlib.pyplot as plt
from cs231n.classifiers.fc_net import *
from cs231n.data_utils import get_CIFAR10_data
from cs231n.gradient_check import eval_numerical_gradient, \
    eval_numerical_gradient_array
from cs231n.solver import Solver

%matplotlib inline
plt.rcParams["figure.figsize"] = (10.0, 8.0) # Set default size of plots.
plt.rcParams["image.interpolation"] = "nearest"
plt.rcParams["image.cmap"] = "gray"

%load_ext autoreload
%autoreload 2

def rel_error(x, y):
    """Returns relative error."""
    return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))
```

```
[4]: # Load the (preprocessed) CIFAR-10 data.
data = get_CIFAR10_data()
for k, v in list(data.items()):
    print(f"{k}: {v.shape}")
```

```
X_train: (49000, 3, 32, 32)
y_train: (49000,)
X_val: (1000, 3, 32, 32)
y_val: (1000,)
X_test: (1000, 3, 32, 32)
y_test: (1000,)
```

1.1 Initial Loss and Gradient Check

As a sanity check, run the following to check the initial loss and to gradient check the network both with and without regularization. This is a good way to see if the initial losses seem reasonable.

For gradient checking, you should expect to see errors around $1e-7$ or less.

```
[6]: np.random.seed(231)
N, D, H1, H2, C = 2, 15, 20, 30, 10
X = np.random.randn(N, D)
y = np.random.randint(C, size=(N,))
```

```

for reg in [0, 3.14]:
    print("Running check with reg = ", reg)
    model = FullyConnectedNet(
        [H1, H2],
        input_dim=D,
        num_classes=C,
        reg=reg,
        weight_scale=5e-2,
        dtype=np.float64
    )

    loss, grads = model.loss(X, y)
    print("Initial loss: ", loss)

    # Most of the errors should be on the order of e-7 or smaller.
    # NOTE: It is fine however to see an error for W2 on the order of e-5
    # for the check when reg = 0.0
    for name in sorted(grads):
        f = lambda _: model.loss(X, y)[0]
        grad_num = eval_numerical_gradient(f, model.params[name],
        ↪ verbose=False, h=1e-5)
        print(f"{name} relative error: {rel_error(grad_num, grads[name])}")

```

```

Running check with reg = 0
Initial loss: 2.300479089768492
W1 relative error: 1.0252674471656573e-07
W2 relative error: 2.2120479295080622e-05
W3 relative error: 4.5623278736665505e-07
b1 relative error: 4.6600944653202505e-09
b2 relative error: 2.085654276112763e-09
b3 relative error: 1.689724888469736e-10
Running check with reg = 3.14
Initial loss: 7.052114776533016
W1 relative error: 3.904541941902138e-09
W2 relative error: 6.86942277940646e-08
W3 relative error: 3.483989247437803e-08
b1 relative error: 1.475242847895799e-08
b2 relative error: 1.7223751746766738e-09
b3 relative error: 1.3200479211447775e-10

```

As another sanity check, make sure your network can overfit on a small dataset of 50 images. First, we will try a three-layer network with 100 units in each hidden layer. In the following cell, tweak the **learning rate** and **weight initialization scale** to overfit and achieve 100% training accuracy within 20 epochs.

```
[10]: # TODO: Use a three-layer Net to overfit 50 training examples by  
# tweaking just the learning rate and initialization scale.
```

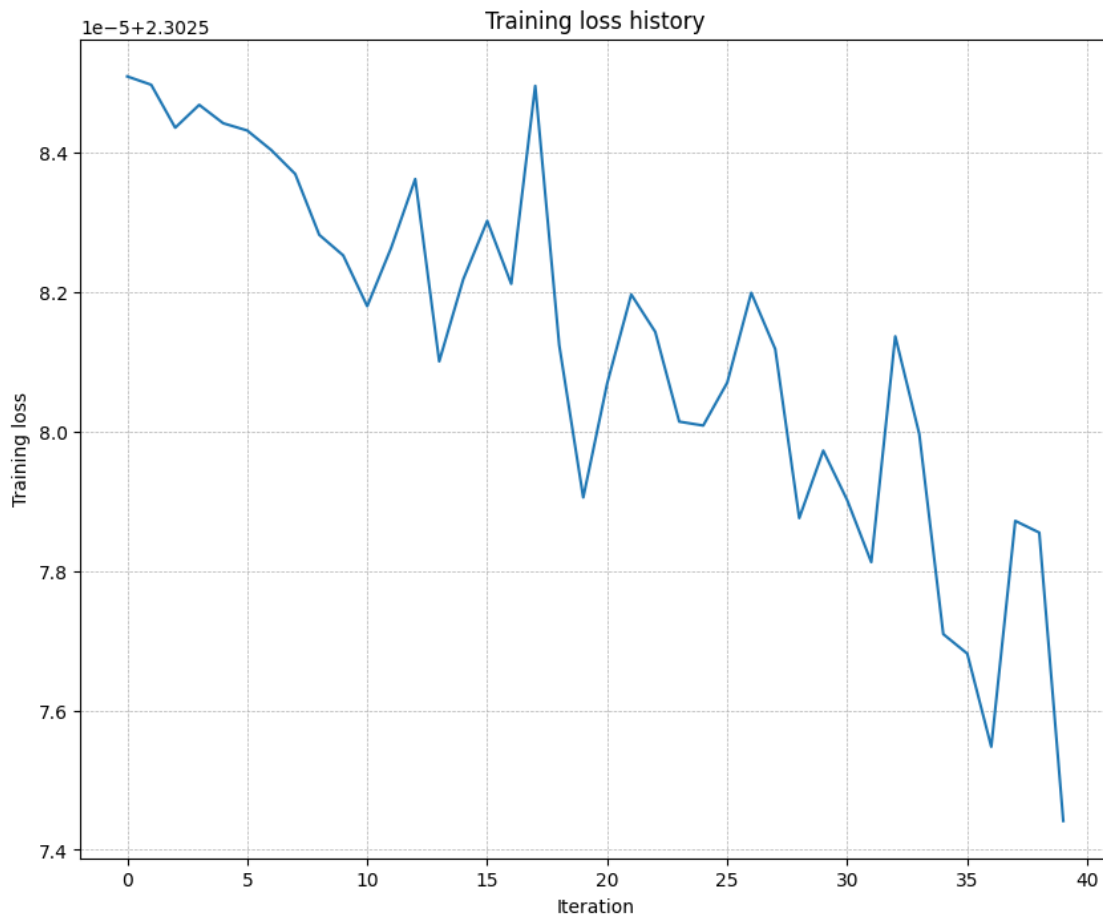
```
num_train = 50  
small_data = {  
    "X_train": data["X_train"][:num_train],  
    "y_train": data["y_train"][:num_train],  
    "X_val": data["X_val"],  
    "y_val": data["y_val"],  
}  
  
weight_scale = 1e-4    # Experiment with this!  
learning_rate = 1e-5   # Experiment with this!  
  
model = FullyConnectedNet(  
    [100, 100],  
    weight_scale=weight_scale,  
    dtype=np.float64  
)  
solver = Solver(  
    model,  
    small_data,  
    print_every=10,  
    num_epochs=20,  
    batch_size=25,  
    update_rule="sgd",  
    optim_config={"learning_rate": learning_rate},  
)  
solver.train()  
  
plt.plot(solver.loss_history)  
plt.title("Training loss history")  
plt.xlabel("Iteration")  
plt.ylabel("Training loss")  
plt.grid(linestyle='--', linewidth=0.5)  
plt.show()
```

```
(Iteration 1 / 40) loss: 2.302585  
(Epoch 0 / 20) train acc: 0.120000; val_acc: 0.119000  
(Epoch 1 / 20) train acc: 0.080000; val_acc: 0.111000  
(Epoch 2 / 20) train acc: 0.160000; val_acc: 0.112000  
(Epoch 3 / 20) train acc: 0.160000; val_acc: 0.112000  
(Epoch 4 / 20) train acc: 0.160000; val_acc: 0.112000  
(Epoch 5 / 20) train acc: 0.160000; val_acc: 0.079000  
(Iteration 11 / 40) loss: 2.302582  
(Epoch 6 / 20) train acc: 0.160000; val_acc: 0.079000
```

```

(Epoch 7 / 20) train acc: 0.160000; val_acc: 0.079000
(Epoch 8 / 20) train acc: 0.160000; val_acc: 0.079000
(Epoch 9 / 20) train acc: 0.160000; val_acc: 0.079000
(Epoch 10 / 20) train acc: 0.160000; val_acc: 0.079000
(Iteration 21 / 40) loss: 2.302581
(Epoch 11 / 20) train acc: 0.160000; val_acc: 0.079000
(Epoch 12 / 20) train acc: 0.160000; val_acc: 0.079000
(Epoch 13 / 20) train acc: 0.160000; val_acc: 0.079000
(Epoch 14 / 20) train acc: 0.160000; val_acc: 0.079000
(Epoch 15 / 20) train acc: 0.160000; val_acc: 0.079000
(Iteration 31 / 40) loss: 2.302579
(Epoch 16 / 20) train acc: 0.160000; val_acc: 0.079000
(Epoch 17 / 20) train acc: 0.160000; val_acc: 0.079000
(Epoch 18 / 20) train acc: 0.160000; val_acc: 0.079000
(Epoch 19 / 20) train acc: 0.160000; val_acc: 0.079000
(Epoch 20 / 20) train acc: 0.160000; val_acc: 0.079000

```



Now, try to use a five-layer network with 100 units on each layer to overfit on 50 training examples. Again, you will have to adjust the learning rate and weight initialization scale, but you should be

able to achieve 100% training accuracy within 20 epochs.

```
[15]: # TODO: Use a five-layer Net to overfit 50 training examples by  
# tweaking just the learning rate and initialization scale.
```

```
num_train = 50
small_data = {
    'X_train': data['X_train'][:num_train],
    'y_train': data['y_train'][:num_train],
    'X_val': data['X_val'],
    'y_val': data['y_val'],
}

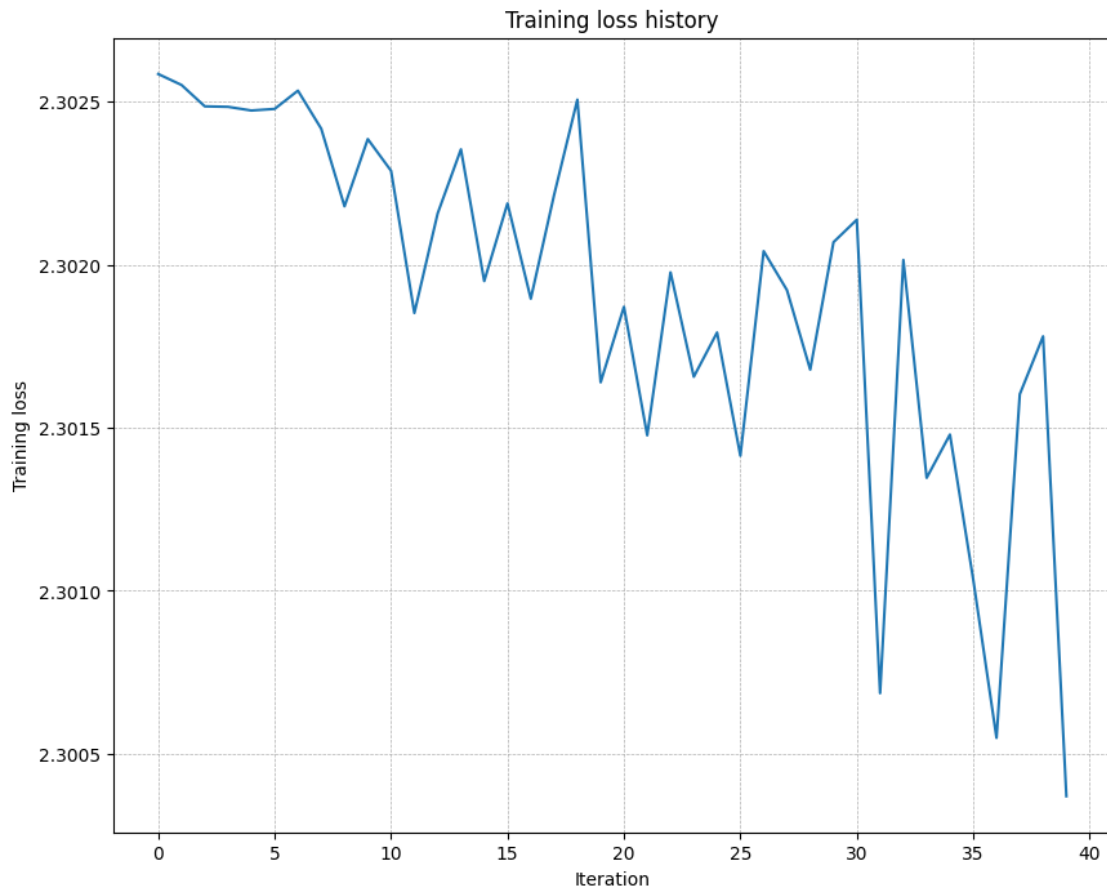
learning_rate = 2e-3 # Experiment with this!
weight_scale = 1e-9 # Experiment with this!

model = FullyConnectedNet(
    [100, 100, 100, 100],
    weight_scale=weight_scale,
    dtype=np.float64
)
solver = Solver(
    model,
    small_data,
    print_every=10,
    num_epochs=20,
    batch_size=25,
    update_rule='sgd',
    optim_config={'learning_rate': learning_rate},
)
solver.train()

plt.plot(solver.loss_history)
plt.title('Training loss history')
plt.xlabel('Iteration')
plt.ylabel('Training loss')
plt.grid(linestyle='--', linewidth=0.5)
plt.show()
```

```
(Iteration 1 / 40) loss: 2.302585
(Epoch 0 / 20) train acc: 0.160000; val_acc: 0.079000
(Epoch 1 / 20) train acc: 0.160000; val_acc: 0.079000
(Epoch 2 / 20) train acc: 0.160000; val_acc: 0.079000
(Epoch 3 / 20) train acc: 0.160000; val_acc: 0.079000
(Epoch 4 / 20) train acc: 0.160000; val_acc: 0.079000
(Epoch 5 / 20) train acc: 0.160000; val_acc: 0.079000
(Iteration 11 / 40) loss: 2.302288
```

(Epoch 6 / 20) train acc: 0.160000; val_acc: 0.079000
(Epoch 7 / 20) train acc: 0.160000; val_acc: 0.079000
(Epoch 8 / 20) train acc: 0.160000; val_acc: 0.079000
(Epoch 9 / 20) train acc: 0.160000; val_acc: 0.079000
(Epoch 10 / 20) train acc: 0.160000; val_acc: 0.079000
(Iteration 21 / 40) loss: 2.301871
(Epoch 11 / 20) train acc: 0.160000; val_acc: 0.079000
(Epoch 12 / 20) train acc: 0.160000; val_acc: 0.079000
(Epoch 13 / 20) train acc: 0.160000; val_acc: 0.079000
(Epoch 14 / 20) train acc: 0.160000; val_acc: 0.079000
(Epoch 15 / 20) train acc: 0.160000; val_acc: 0.079000
(Iteration 31 / 40) loss: 2.302138
(Epoch 16 / 20) train acc: 0.160000; val_acc: 0.079000
(Epoch 17 / 20) train acc: 0.160000; val_acc: 0.079000
(Epoch 18 / 20) train acc: 0.160000; val_acc: 0.079000
(Epoch 19 / 20) train acc: 0.160000; val_acc: 0.079000
(Epoch 20 / 20) train acc: 0.160000; val_acc: 0.079000



1.2 Inline Question 1:

Did you notice anything about the comparative difficulty of training the three-layer network vs. training the five-layer network? In particular, based on your experience, which network seemed more sensitive to the initialization scale? Why do you think that is the case?

1.3 Answer:

With a shallow network, gradients only pass through a few layers, so the effects of a poor initialization are relatively limited. In the five-layer network, however, activations and gradients are repeatedly multiplied through more layers.

2 Update rules

So far we have used vanilla stochastic gradient descent (SGD) as our update rule. More sophisticated update rules can make it easier to train deep networks. We will implement a few of the most commonly used update rules and compare them to vanilla SGD.

2.1 SGD+Momentum

Stochastic gradient descent with momentum is a widely used update rule that tends to make deep networks converge faster than vanilla stochastic gradient descent. See the Momentum Update section at <http://cs231n.github.io/neural-networks-3/#sgd> for more information.

Open the file `cs231n/optim.py` and read the documentation at the top of the file to make sure you understand the API. Implement the SGD+momentum update rule in the function `sgd_momentum` and run the following to check your implementation. You should see errors less than $e-8$.

```
[16]: from cs231n.optim import sgd_momentum

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N*D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N*D).reshape(N, D)
v = np.linspace(0.6, 0.9, num=N*D).reshape(N, D)

config = {"learning_rate": 1e-3, "velocity": v}
next_w, _ = sgd_momentum(w, dw, config=config)

expected_next_w = np.asarray([
    [ 0.1406,      0.20738947,  0.27417895,  0.34096842,  0.40775789],
    [ 0.47454737,  0.54133684,  0.60812632,  0.67491579,  0.74170526],
    [ 0.80849474,  0.87528421,  0.94207368,  1.00886316,  1.07565263],
    [ 1.14244211,  1.20923158,  1.27602105,  1.34281053,  1.4096    ]])
expected_velocity = np.asarray([
    [ 0.5406,      0.55475789,  0.56891579,  0.58307368,  0.59723158],
    [ 0.61138947,  0.62554737,  0.63970526,  0.65386316,  0.66802105],
    [ 0.68217895,  0.69633684,  0.71049474,  0.72465263,  0.73881053],
    [ 0.75296842,  0.76712632,  0.78128421,  0.79544211,  0.8096    ]])
```

```
# Should see relative errors around e-8 or less
print("next_w error: ", rel_error(next_w, expected_next_w))
print("velocity error: ", rel_error(expected_velocity, config["velocity"]))
```

```
next_w error:  8.882347033505819e-09
velocity error: 4.269287743278663e-09
```

Once you have done so, run the following to train a six-layer network with both SGD and SGD+momentum. You should see the SGD+momentum update rule converge faster.

```
[17]: num_train = 4000
small_data = {
    'X_train': data['X_train'][:num_train],
    'y_train': data['y_train'][:num_train],
    'X_val': data['X_val'],
    'y_val': data['y_val'],
}

solvers = {}

for update_rule in ['sgd', 'sgd_momentum']:
    print('Running with ', update_rule)
    model = FullyConnectedNet(
        [100, 100, 100, 100, 100],
        weight_scale=5e-2
    )

    solver = Solver(
        model,
        small_data,
        num_epochs=5,
        batch_size=100,
        update_rule=update_rule,
        optim_config={'learning_rate': 5e-3},
        verbose=True,
    )
    solvers[update_rule] = solver
    solver.train()

fig, axes = plt.subplots(3, 1, figsize=(15, 15))

axes[0].set_title('Training loss')
axes[0].set_xlabel('Iteration')
axes[1].set_title('Training accuracy')
axes[1].set_xlabel('Epoch')
axes[2].set_title('Validation accuracy')
axes[2].set_xlabel('Epoch')
```



```

for update_rule, solver in solvers.items():
    axes[0].plot(solver.loss_history, label=f"loss_{update_rule}")
    axes[1].plot(solver.train_acc_history, label=f"train_acc_{update_rule}")
    axes[2].plot(solver.val_acc_history, label=f"val_acc_{update_rule}")

for ax in axes:
    ax.legend(loc="best", ncol=4)
    ax.grid(linestyle='--', linewidth=0.5)

plt.show()

```

Running with `sgd`

```

(Iteration 1 / 200) loss: 2.781483
(Epoch 0 / 5) train acc: 0.102000; val_acc: 0.103000
(Iteration 11 / 200) loss: 2.253357
(Iteration 21 / 200) loss: 2.227992
(Iteration 31 / 200) loss: 2.117558
(Epoch 1 / 5) train acc: 0.243000; val_acc: 0.221000
(Iteration 41 / 200) loss: 2.167535
(Iteration 51 / 200) loss: 2.119403
(Iteration 61 / 200) loss: 2.051367
(Iteration 71 / 200) loss: 2.007986
(Epoch 2 / 5) train acc: 0.295000; val_acc: 0.258000
(Iteration 81 / 200) loss: 1.990176
(Iteration 91 / 200) loss: 2.013758
(Iteration 101 / 200) loss: 1.922855
(Iteration 111 / 200) loss: 2.012823
(Epoch 3 / 5) train acc: 0.309000; val_acc: 0.297000
(Iteration 121 / 200) loss: 2.001308
(Iteration 131 / 200) loss: 1.839431
(Iteration 141 / 200) loss: 1.742192
(Iteration 151 / 200) loss: 1.834244
(Epoch 4 / 5) train acc: 0.333000; val_acc: 0.309000
(Iteration 161 / 200) loss: 1.884877
(Iteration 171 / 200) loss: 1.829726
(Iteration 181 / 200) loss: 1.900484
(Iteration 191 / 200) loss: 1.921633
(Epoch 5 / 5) train acc: 0.370000; val_acc: 0.307000

```

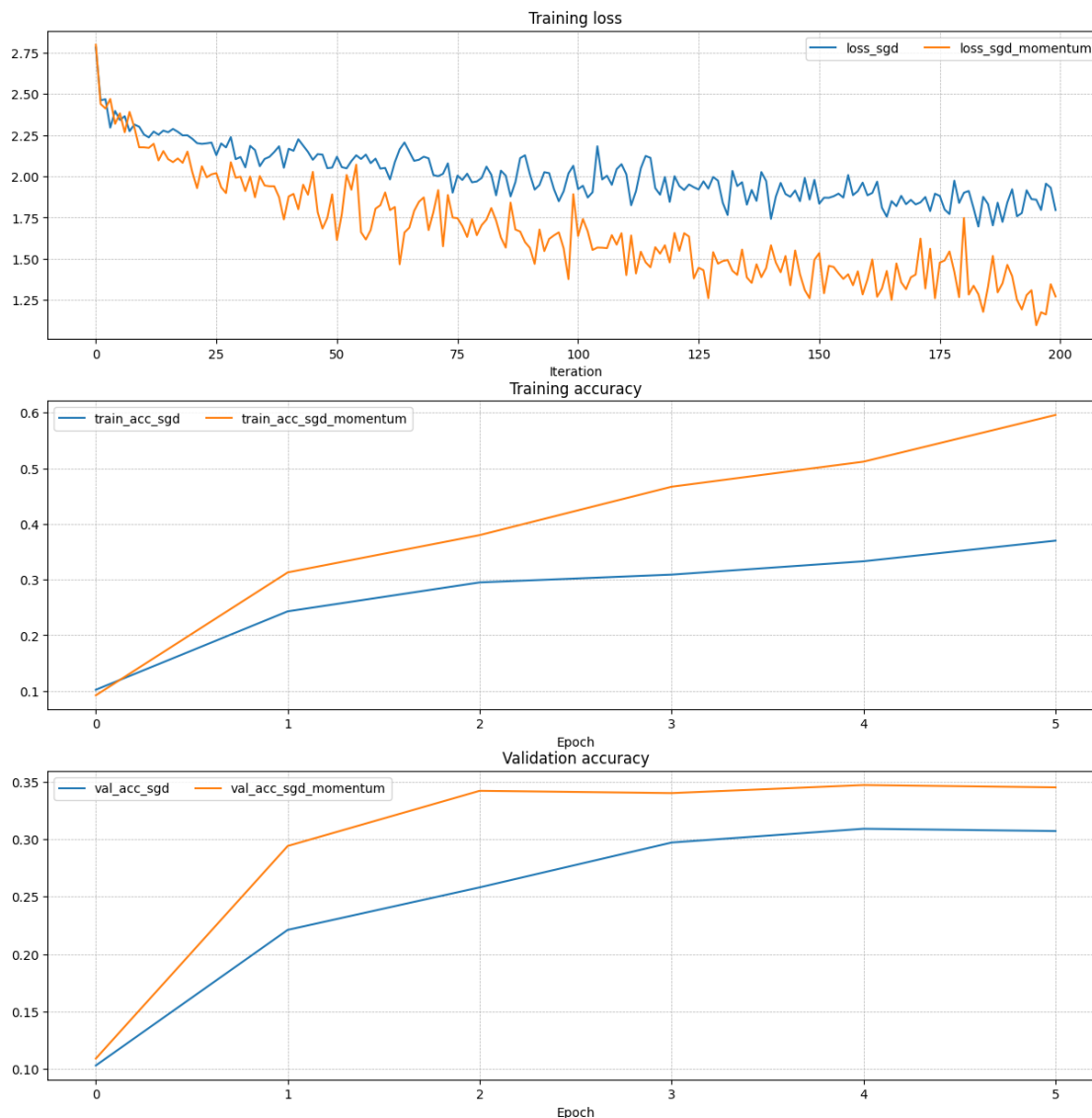
Running with `sgd_momentum`

```

(Iteration 1 / 200) loss: 2.798030
(Epoch 0 / 5) train acc: 0.092000; val_acc: 0.109000
(Iteration 11 / 200) loss: 2.175850
(Iteration 21 / 200) loss: 2.026721
(Iteration 31 / 200) loss: 1.998088
(Epoch 1 / 5) train acc: 0.313000; val_acc: 0.294000
(Iteration 41 / 200) loss: 1.878105
(Iteration 51 / 200) loss: 1.614165

```

(Iteration 61 / 200) loss: 1.902253
(Iteration 71 / 200) loss: 1.783638
(Epoch 2 / 5) train acc: 0.380000; val_acc: 0.342000
(Iteration 81 / 200) loss: 1.703756
(Iteration 91 / 200) loss: 1.566741
(Iteration 101 / 200) loss: 1.638095
(Iteration 111 / 200) loss: 1.401242
(Epoch 3 / 5) train acc: 0.467000; val_acc: 0.340000
(Iteration 121 / 200) loss: 1.656869
(Iteration 131 / 200) loss: 1.486296
(Iteration 141 / 200) loss: 1.581910
(Iteration 151 / 200) loss: 1.534362
(Epoch 4 / 5) train acc: 0.512000; val_acc: 0.347000
(Iteration 161 / 200) loss: 1.374730
(Iteration 171 / 200) loss: 1.404577
(Iteration 181 / 200) loss: 1.747787
(Iteration 191 / 200) loss: 1.394705
(Epoch 5 / 5) train acc: 0.596000; val_acc: 0.345000



2.2 RMSProp and Adam

RMSProp [1] and Adam [2] are update rules that set per-parameter learning rates by using a running average of the second moments of gradients.

In the file `cs231n/optim.py`, implement the RMSProp update rule in the `rmsprop` function and implement the Adam update rule in the `adam` function, and check your implementations using the tests below.

NOTE: Please implement the *complete* Adam update rule (with the bias correction mechanism), not the first simplified version mentioned in the course notes.

[1] Tijmen Tieleman and Geoffrey Hinton. “Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude.” COURSE: Neural Networks for Machine Learning 4 (2012).

[2] Diederik Kingma and Jimmy Ba, “Adam: A Method for Stochastic Optimization”, ICLR 2015.

```
[18]: # Test RMSProp implementation
from cs231n.optim import rmsprop

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N*D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N*D).reshape(N, D)
cache = np.linspace(0.6, 0.9, num=N*D).reshape(N, D)

config = {'learning_rate': 1e-2, 'cache': cache}
next_w, _ = rmsprop(w, dw, config=config)

expected_next_w = np.asarray([
    [-0.39223849, -0.34037513, -0.28849239, -0.23659121, -0.18467247],
    [-0.132737,   -0.08078555, -0.02881884,  0.02316247,  0.07515774],
    [ 0.12716641,  0.17918792,  0.23122175,  0.28326742,  0.33532447],
    [ 0.38739248,  0.43947102,  0.49155973,  0.54365823,  0.59576619]])
expected_cache = np.asarray([
    [ 0.5976,      0.6126277,  0.6277108,  0.64284931,  0.65804321],
    [ 0.67329252,  0.68859723,  0.70395734,  0.71937285,  0.73484377],
    [ 0.75037008,  0.7659518,   0.78158892,  0.79728144,  0.81302936],
    [ 0.82883269,  0.84469141,  0.86060554,  0.87657507,  0.8926   ]])

# You should see relative errors around e-7 or less
print('next_w error: ', rel_error(expected_next_w, next_w))
print('cache error: ', rel_error(expected_cache, config['cache']))
```

next_w error: 9.524687511038133e-08

cache error: 2.6477955807156126e-09

```
[19]: # Test Adam implementation
from cs231n.optim import adam

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N*D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N*D).reshape(N, D)
m = np.linspace(0.6, 0.9, num=N*D).reshape(N, D)
v = np.linspace(0.7, 0.5, num=N*D).reshape(N, D)

config = {'learning_rate': 1e-2, 'm': m, 'v': v, 't': 5}
next_w, _ = adam(w, dw, config=config)

expected_next_w = np.asarray([
    [-0.40094747, -0.34836187, -0.29577703, -0.24319299, -0.19060977],
    [-0.1380274,  -0.08544591, -0.03286534,  0.01971428,  0.0722929],
    [ 0.1248705,   0.17744702,  0.23002243,  0.28259667,  0.33516969],
    [ 0.38774145,  0.44031188,  0.49288093,  0.54544852,  0.59801459]])
```

```

expected_v = np.asarray([
    [ 0.69966,      0.68908382,  0.67851319,  0.66794809,  0.65738853,],
    [ 0.64683452,  0.63628604,  0.6257431,   0.61520571,  0.60467385,],
    [ 0.59414753,  0.58362676,  0.57311152,  0.56260183,  0.55209767,],
    [ 0.54159906,  0.53110598,  0.52061845,  0.51013645,  0.49966,   ]])
expected_m = np.asarray([
    [ 0.48,          0.49947368,  0.51894737,  0.53842105,  0.55789474],
    [ 0.57736842,  0.59684211,  0.61631579,  0.63578947,  0.65526316],
    [ 0.67473684,  0.69421053,  0.71368421,  0.73315789,  0.75263158],
    [ 0.77210526,  0.79157895,  0.81105263,  0.83052632,  0.85         ]])

# You should see relative errors around e-7 or less
print('next_w error: ', rel_error(expected_next_w, next_w))
print('v error: ', rel_error(expected_v, config['v']))
print('m error: ', rel_error(expected_m, config['m']))

```

```

next_w error:  1.1395691798535431e-07
v error:  4.208314038113071e-09
m error:  4.214963193114416e-09

```

Once you have debugged your RMSProp and Adam implementations, run the following to train a pair of deep networks using these new update rules:

```

[20]: learning_rates = {'rmsprop': 1e-4, 'adam': 1e-3}
for update_rule in ['adam', 'rmsprop']:
    print('Running with ', update_rule)
    model = FullyConnectedNet(
        [100, 100, 100, 100, 100],
        weight_scale=5e-2
    )
    solver = Solver(
        model,
        small_data,
        num_epochs=5,
        batch_size=100,
        update_rule=update_rule,
        optim_config={'learning_rate': learning_rates[update_rule]},
        verbose=True
    )
    solvers[update_rule] = solver
    solver.train()
    print()

fig, axes = plt.subplots(3, 1, figsize=(15, 15))

axes[0].set_title('Training loss')
axes[0].set_xlabel('Iteration')
axes[1].set_title('Training accuracy')

```

```

axes[1].set_xlabel('Epoch')
axes[2].set_title('Validation accuracy')
axes[2].set_xlabel('Epoch')

for update_rule, solver in solvers.items():
    axes[0].plot(solver.loss_history, label=f"{update_rule}")
    axes[1].plot(solver.train_acc_history, label=f"{update_rule}")
    axes[2].plot(solver.val_acc_history, label=f"{update_rule}")

for ax in axes:
    ax.legend(loc='best', ncol=4)
    ax.grid(linestyle='--', linewidth=0.5)

plt.show()

```

Running with adam

```

(Iteration 1 / 200) loss: 3.516127
(Epoch 0 / 5) train acc: 0.159000; val_acc: 0.146000
(Iteration 11 / 200) loss: 2.094091
(Iteration 21 / 200) loss: 1.981708
(Iteration 31 / 200) loss: 1.879446
(Epoch 1 / 5) train acc: 0.323000; val_acc: 0.304000
(Iteration 41 / 200) loss: 1.679698
(Iteration 51 / 200) loss: 1.863699
(Iteration 61 / 200) loss: 1.777174
(Iteration 71 / 200) loss: 1.358578
(Epoch 2 / 5) train acc: 0.420000; val_acc: 0.359000
(Iteration 81 / 200) loss: 1.644620
(Iteration 91 / 200) loss: 1.426451
(Iteration 101 / 200) loss: 1.615922
(Iteration 111 / 200) loss: 1.532247
(Epoch 3 / 5) train acc: 0.482000; val_acc: 0.366000
(Iteration 121 / 200) loss: 1.428785
(Iteration 131 / 200) loss: 1.494473
(Iteration 141 / 200) loss: 1.310759
(Iteration 151 / 200) loss: 1.727774
(Epoch 4 / 5) train acc: 0.568000; val_acc: 0.367000
(Iteration 161 / 200) loss: 1.241827
(Iteration 171 / 200) loss: 1.342611
(Iteration 181 / 200) loss: 1.228294
(Iteration 191 / 200) loss: 1.104616
(Epoch 5 / 5) train acc: 0.578000; val_acc: 0.396000

```

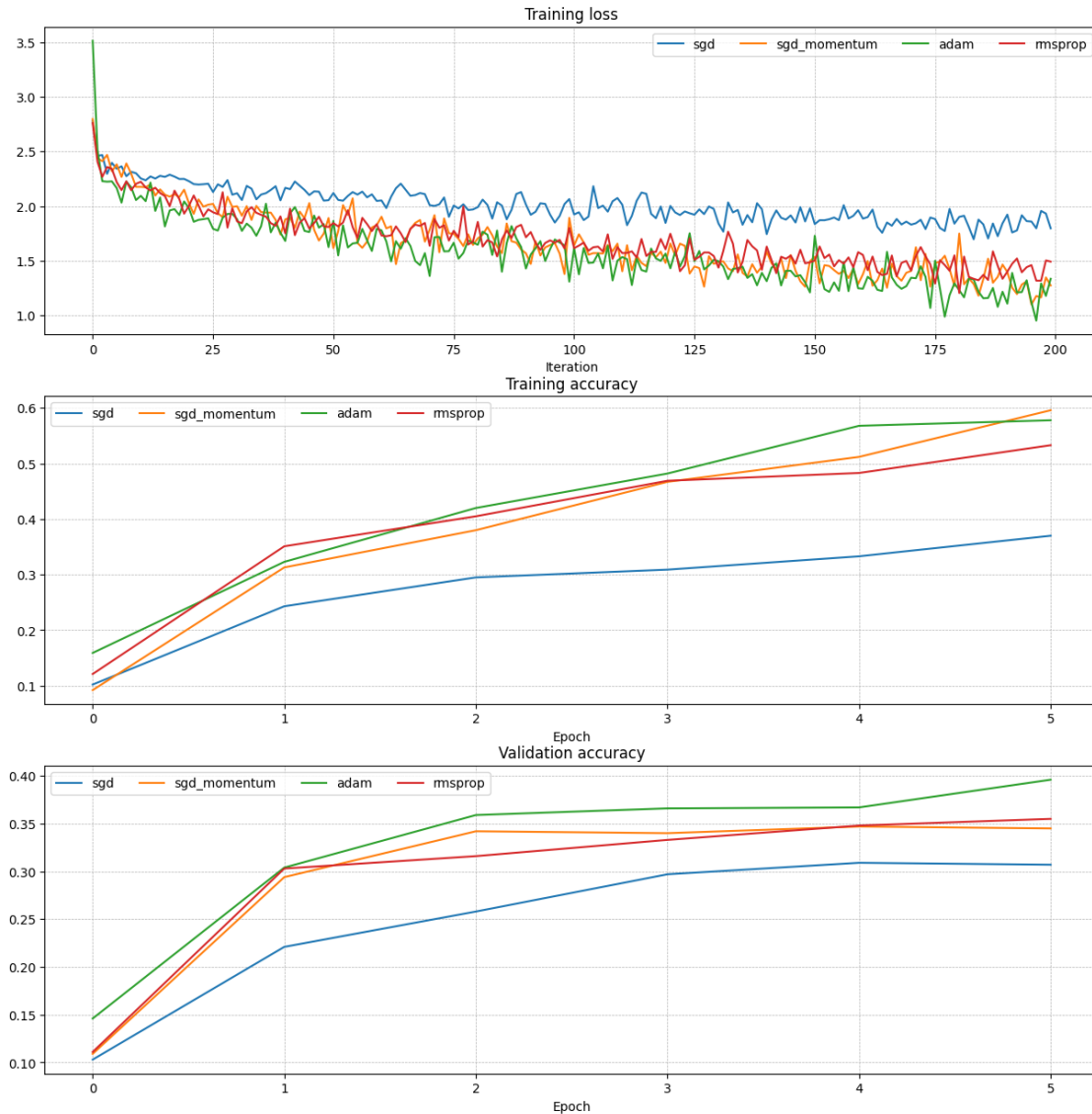
Running with rmsprop

```

(Iteration 1 / 200) loss: 2.760167
(Epoch 0 / 5) train acc: 0.121000; val_acc: 0.111000
(Iteration 11 / 200) loss: 2.222712

```

(Iteration 21 / 200) loss: 2.001674
(Iteration 31 / 200) loss: 1.859883
(Epoch 1 / 5) train acc: 0.351000; val_acc: 0.303000
(Iteration 41 / 200) loss: 1.749578
(Iteration 51 / 200) loss: 1.843670
(Iteration 61 / 200) loss: 1.729902
(Iteration 71 / 200) loss: 1.635343
(Epoch 2 / 5) train acc: 0.405000; val_acc: 0.316000
(Iteration 81 / 200) loss: 1.855064
(Iteration 91 / 200) loss: 1.713861
(Iteration 101 / 200) loss: 1.613553
(Iteration 111 / 200) loss: 1.568645
(Epoch 3 / 5) train acc: 0.469000; val_acc: 0.333000
(Iteration 121 / 200) loss: 1.746163
(Iteration 131 / 200) loss: 1.434632
(Iteration 141 / 200) loss: 1.628080
(Iteration 151 / 200) loss: 1.499221
(Epoch 4 / 5) train acc: 0.483000; val_acc: 0.348000
(Iteration 161 / 200) loss: 1.551597
(Iteration 171 / 200) loss: 1.408042
(Iteration 181 / 200) loss: 1.202519
(Iteration 191 / 200) loss: 1.398135
(Epoch 5 / 5) train acc: 0.533000; val_acc: 0.355000



2.3 Inline Question 2:

AdaGrad, like Adam, is a per-parameter optimization method that uses the following update rule:

```
cache += dw**2
w += - learning_rate * dw / (np.sqrt(cache) + eps)
```

John notices that when he was training a network with AdaGrad that the updates became very small, and that his network was learning slowly. Using your knowledge of the AdaGrad update rule, why do you think the updates would become very small? Would Adam have the same issue?

2.4 Answer:

In AdaGrad, the cache is the cumulative sum of squared gradients. Since this sum keeps growing over time and is never decayed, the denominator becomes larger and larger.

Instead, it keeps an exponentially decaying moving average of both the gradients and the squared gradients.

3 Train a Good Model!

Train the best fully connected model that you can on CIFAR-10, storing your best model in the `best_model` variable. We require you to get at least 50% accuracy on the validation set using a fully connected network.

If you are careful it should be possible to get accuracies above 55%, but we don't require it for this part and won't assign extra credit for doing so. Later in the next assignment, we will ask you to train the best convolutional network that you can on CIFAR-10, and we would prefer that you spend your effort working on convolutional networks rather than fully connected networks.

Note: In the next assignment, you will learn techniques like BatchNormalization and Dropout which can help you train powerful models.

```
[21]: best_model = None

#####
# TODO: Train the best FullyConnectedNet that you can on CIFAR-10. You might #
# find batch/layer normalization and dropout useful. Store your best model in #
# the best_model variable.                                                    #
#####
# *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
best_model = FullyConnectedNet(
    [512, 256, 128],
    input_dim=3 * 32 * 32,
    num_classes=10,
    dropout_keep_ratio=0.75,
    normalization="batchnorm",
    reg=1e-3,
    weight_scale=2e-2,
    dtype=np.float32,
)

solver = Solver(
    best_model,
    data,
    update_rule="adam",
    optim_config={
        "learning_rate": 1e-3,
    },
    lr_decay=0.95,
```

```

    num_epochs=20,
    batch_size=100,
    print_every=100,
    verbose=True,
)

solver.train()

best_model = solver.model

# *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
#####
#                                     END OF YOUR CODE                                     #
#####

(Iteration 1 / 9800) loss: 2.658996
(Epoch 0 / 20) train acc: 0.126000; val_acc: 0.174000
(Iteration 101 / 9800) loss: 1.947294
(Iteration 201 / 9800) loss: 1.939011
(Iteration 301 / 9800) loss: 1.987434
(Iteration 401 / 9800) loss: 1.747098
(Epoch 1 / 20) train acc: 0.447000; val_acc: 0.468000
(Iteration 501 / 9800) loss: 1.696628
(Iteration 601 / 9800) loss: 1.623697
(Iteration 701 / 9800) loss: 1.895379
(Iteration 801 / 9800) loss: 1.700321
(Iteration 901 / 9800) loss: 1.750418
(Epoch 2 / 20) train acc: 0.480000; val_acc: 0.476000
(Iteration 1001 / 9800) loss: 1.676506
(Iteration 1101 / 9800) loss: 1.791918
(Iteration 1201 / 9800) loss: 1.652445
(Iteration 1301 / 9800) loss: 1.753721
(Iteration 1401 / 9800) loss: 1.979808
(Epoch 3 / 20) train acc: 0.514000; val_acc: 0.482000
(Iteration 1501 / 9800) loss: 1.708324
(Iteration 1601 / 9800) loss: 1.662180
(Iteration 1701 / 9800) loss: 1.606133
(Iteration 1801 / 9800) loss: 1.552473
(Iteration 1901 / 9800) loss: 1.611714
(Epoch 4 / 20) train acc: 0.491000; val_acc: 0.501000
(Iteration 2001 / 9800) loss: 1.694525
(Iteration 2101 / 9800) loss: 1.579711
(Iteration 2201 / 9800) loss: 1.680901
(Iteration 2301 / 9800) loss: 1.492593
(Iteration 2401 / 9800) loss: 1.705753
(Epoch 5 / 20) train acc: 0.515000; val_acc: 0.478000
(Iteration 2501 / 9800) loss: 1.612318

```

(Iteration 2601 / 9800) loss: 1.510763
(Iteration 2701 / 9800) loss: 1.706220
(Iteration 2801 / 9800) loss: 1.601159
(Iteration 2901 / 9800) loss: 1.666646
(Epoch 6 / 20) train acc: 0.504000; val_acc: 0.513000
(Iteration 3001 / 9800) loss: 1.371879
(Iteration 3101 / 9800) loss: 1.581917
(Iteration 3201 / 9800) loss: 1.782006
(Iteration 3301 / 9800) loss: 1.521989
(Iteration 3401 / 9800) loss: 1.393249
(Epoch 7 / 20) train acc: 0.557000; val_acc: 0.524000
(Iteration 3501 / 9800) loss: 1.560966
(Iteration 3601 / 9800) loss: 1.508661
(Iteration 3701 / 9800) loss: 1.465114
(Iteration 3801 / 9800) loss: 1.569618
(Iteration 3901 / 9800) loss: 1.542510
(Epoch 8 / 20) train acc: 0.542000; val_acc: 0.517000
(Iteration 4001 / 9800) loss: 1.454485
(Iteration 4101 / 9800) loss: 1.553035
(Iteration 4201 / 9800) loss: 1.595341
(Iteration 4301 / 9800) loss: 1.553531
(Iteration 4401 / 9800) loss: 1.543081
(Epoch 9 / 20) train acc: 0.577000; val_acc: 0.533000
(Iteration 4501 / 9800) loss: 1.556193
(Iteration 4601 / 9800) loss: 1.448881
(Iteration 4701 / 9800) loss: 1.467752
(Iteration 4801 / 9800) loss: 1.325934
(Epoch 10 / 20) train acc: 0.549000; val_acc: 0.517000
(Iteration 4901 / 9800) loss: 1.488296
(Iteration 5001 / 9800) loss: 1.556532
(Iteration 5101 / 9800) loss: 1.691036
(Iteration 5201 / 9800) loss: 1.506335
(Iteration 5301 / 9800) loss: 1.529362
(Epoch 11 / 20) train acc: 0.555000; val_acc: 0.553000
(Iteration 5401 / 9800) loss: 1.551033
(Iteration 5501 / 9800) loss: 1.605537
(Iteration 5601 / 9800) loss: 1.681810
(Iteration 5701 / 9800) loss: 1.689008
(Iteration 5801 / 9800) loss: 1.682611
(Epoch 12 / 20) train acc: 0.581000; val_acc: 0.549000
(Iteration 5901 / 9800) loss: 1.557838
(Iteration 6001 / 9800) loss: 1.518531
(Iteration 6101 / 9800) loss: 1.778926
(Iteration 6201 / 9800) loss: 1.439621
(Iteration 6301 / 9800) loss: 1.384154
(Epoch 13 / 20) train acc: 0.612000; val_acc: 0.553000
(Iteration 6401 / 9800) loss: 1.692613
(Iteration 6501 / 9800) loss: 1.516852

```

(Iteration 6601 / 9800) loss: 1.608402
(Iteration 6701 / 9800) loss: 1.419053
(Iteration 6801 / 9800) loss: 1.590062
(Epoch 14 / 20) train acc: 0.590000; val_acc: 0.567000
(Iteration 6901 / 9800) loss: 1.571020
(Iteration 7001 / 9800) loss: 1.567008
(Iteration 7101 / 9800) loss: 1.603729
(Iteration 7201 / 9800) loss: 1.451014
(Iteration 7301 / 9800) loss: 1.492034
(Epoch 15 / 20) train acc: 0.608000; val_acc: 0.571000
(Iteration 7401 / 9800) loss: 1.425700
(Iteration 7501 / 9800) loss: 1.304377
(Iteration 7601 / 9800) loss: 1.318662
(Iteration 7701 / 9800) loss: 1.672438
(Iteration 7801 / 9800) loss: 1.557974
(Epoch 16 / 20) train acc: 0.637000; val_acc: 0.567000
(Iteration 7901 / 9800) loss: 1.491879
(Iteration 8001 / 9800) loss: 1.376770
(Iteration 8101 / 9800) loss: 1.387244
(Iteration 8201 / 9800) loss: 1.367975
(Iteration 8301 / 9800) loss: 1.299722
(Epoch 17 / 20) train acc: 0.619000; val_acc: 0.569000
(Iteration 8401 / 9800) loss: 1.421649
(Iteration 8501 / 9800) loss: 1.440571
(Iteration 8601 / 9800) loss: 1.332318
(Iteration 8701 / 9800) loss: 1.216739
(Iteration 8801 / 9800) loss: 1.292117
(Epoch 18 / 20) train acc: 0.642000; val_acc: 0.563000
(Iteration 8901 / 9800) loss: 1.450859
(Iteration 9001 / 9800) loss: 1.694393
(Iteration 9101 / 9800) loss: 1.531892
(Iteration 9201 / 9800) loss: 1.363095
(Iteration 9301 / 9800) loss: 1.492577
(Epoch 19 / 20) train acc: 0.600000; val_acc: 0.577000
(Iteration 9401 / 9800) loss: 1.356258
(Iteration 9501 / 9800) loss: 1.412659
(Iteration 9601 / 9800) loss: 1.555174
(Iteration 9701 / 9800) loss: 1.249631
(Epoch 20 / 20) train acc: 0.655000; val_acc: 0.588000

```

4 Test Your Model!

Run your best model on the validation and test sets. You should achieve at least 50% accuracy on the validation set and the test set.

```

[22]: y_test_pred = np.argmax(best_model.loss(data['X_test']), axis=1)
      y_val_pred = np.argmax(best_model.loss(data['X_val']), axis=1)

```

```
print('Validation set accuracy: ', (y_val_pred == data['y_val']).mean())  
print('Test set accuracy: ', (y_test_pred == data['y_test']).mean())
```

Validation set accuracy: 0.588

Test set accuracy: 0.573